Towards AI That Improves Itself: A Survey of Recursive Self-Improvement

Hanjing Li[1,*] **Qiguang Chen**[2,*] **Chenyuan Zhang**[1,4] **Qionglin Qiu**[1]
Fanqing Meng[3] **Mengkang Hu**[2] **Libo Qin**[1] **Min Zhang**[1]
[1] Harbin Institute of Technology (Shenzhen) [2] University of Hong Kong
[3] National University of Singapore [4] Shanghai Innovation Institute
[*] Equal contribution.

Abstract

Recent advances in agentic systems allow AI systems to conduct experiments, interpret feedback, and modify their own workflows, transforming **Recursive Self-Improvement** (RSI) from a theoretical concept into an engineering problem. However, the field still lacks a unified account of what constitutes RSI and how its improvement loop should be evaluated, making it difficult to distinguish one-off revision, persistent self-evolution, meta-evolution, and genuinely recursive improvement. To address this gap, we formalize the operational boundaries of RSI through four stages: evolution, self-evolution, meta-evolution, and recursive self-improvement. We anchor recursion in state inheritance, where an accepted improvement becomes part of the starting state of a later improvement cycle, and treat *recursive progress* as a stronger claim requiring separate evidence that the inherited state improves the improver itself. Across these stages, we introduce a common **Proposal** $\rightarrow$ **Feedback** $\rightarrow$ **Optimization** loop that covers candidate generation, execution and verification, and the decision to retain, revise, reject, promote, or roll back a change. We then organize RSI-related methods and systems by four evolution targets: behavior, agent systems, models and data, and research processes, while characterizing how the loop governs exploration and evidence quality. Finally, we identify major research gaps and outline future directions across seven dimensions, emphasizing evidence-based acceptance, reuse across improvement cycles, and transfer beyond the tasks or evaluators that produced the gains. This structured overview supports rigorous comparison and guides the development of verifiable, persistent, and increasingly autonomous RSI systems. An accompanying website maintains the living bibliography and updates at `https://github.com/Lee1003-lee/Awesome-RSI-Research`.

1 Introduction

AI systems have traditionally been improved through development processes designed and run by humans, including self-play, synthetic-data generation, and preference optimization [184, 200, 201, 236]. This boundary is beginning to blur as large language models and agentic systems take part in software engineering and machine-learning experiments [23, 100, 230, 275]. Modern agents can write and modify code [230, 275], use external tools [192, 283], and analyze failures using language or tool feedback [78, 198]. Other systems generate training data and update models [236, 334], optimize prompts and agent workflows [98, 307], and propose and execute new experiments [23, 100, 146].

This shift motivates renewed attention to **Recursive Self-Improvement (RSI)** [6, 30, 188]. In this survey, we distinguish four stages of self-improvement: *evolution*, *self-evolution*, *meta-evolution*, and *recursive self-improvement (RSI)*. Evolution improves the current output or trajectory without persistence; self-evolution retains an accepted improvement and reuses it in later tasks or runs; meta-evolution occurs when an accepted change modifies the mechanism that produces, evaluates, selects,

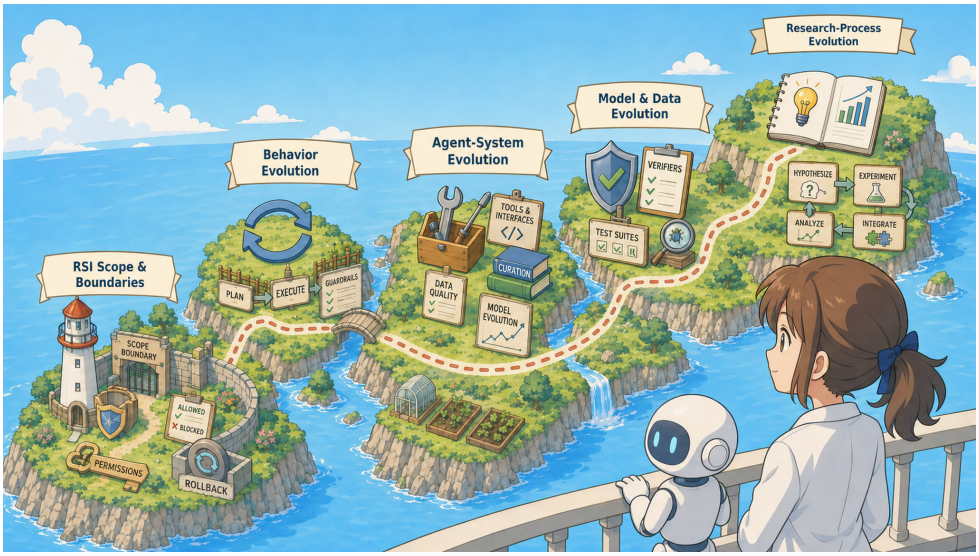

Figure 1: Overview of recursive self-improvement (RSI). The figure maps the survey's scope and boundaries across behavior, agent-system, model-and-data, and research-process evolution, emphasizing the move from local revision toward persistent improvement loops.

or applies subsequent improvements. We reserve RSI for cases in which a later improvement cycle inherits and builds on an improvement accepted and retained by an earlier cycle, rather than restarting from the same base state; this state inheritance is what makes improvement recursive. We use *recursive progress* for the stronger, separately evidenced claim that the inherited state also improves a later improvement procedure, rather than merely improving a subsequent task outcome. Figure 1 summarizes this boundary by contrasting one-off output revision with persistent improvement loops that carry accepted changes into later cycles.

Early discussions of RSI mainly appeared in theories of intelligence explosion and AGI safety [16, 76, 299]. They asked whether continued self-improvement could speed up capability growth and create control risks [16, 172]. Then, several self-evolving agents and autonomous research agents put proposing changes, running experiments, reading feedback, and keeping or rolling back improvements into repeated loops [20, 146, 148, 288, 304, 305]. More recently, a series of works broadens this picture from source rewriting to recursive skill learning, posterior-guided harness updates, release-oriented evaluation, and end-to-end scientific research [77, 125, 147, 234, 250, 252].

As a result, RSI is moving from a thought experiment about future superintelligence toward a practical issue for automating AI R&D, designing agent systems, and forecasting capability growth [49]. Yet most current systems remain bounded rather than fully closed-loop RSI. RSI-related systems today are therefore better understood as exhibiting different combinations of local evolution, persistent self-evolution, meta-evolution, and state-inheriting recursion, with recursive progress requiring stronger evidence [30]. The central thesis of this survey is that modern RSI is bottlenecked not by proposing changes, but by reliably accepting changes that persist and transfer. Specifically, such systems differ in what they can modify, how they verify changes, and whether they can retain them without human approval. Repeated optimization can overfit benchmarks [161], exploit evaluator bias [324], or verifier feedback [78]. Live evaluation tests transfer beyond fixed benchmarks [161, 243].

To make this distinction operational, we organize the literature through two complementary lenses: evolution targets and improvement mechanisms. The latter follows a three-stage loop, **Proposal** $\rightarrow$ **Feedback** $\rightarrow$ **Optimization**: **Proposal** selects a target and generates candidate changes; **Feedback** executes and verifies them through tests, benchmarks, critics, simulators, or human review; and **Optimization** retains, revises, rejects, promotes, or rolls back each change. These lenses are complementary because the same mechanism can improve different targets, while the same target can be improved by different mechanisms.

We therefore ask not only "did the score increase?" but also "what changed, why was it accepted, does it transfer, is it reused in later cycles, and does the retained change improve the process that produces subsequent improvements?" This framework distinguishes bounded self-evolution from recursive self-improvement and highlights the evidence missing from current systems. The key

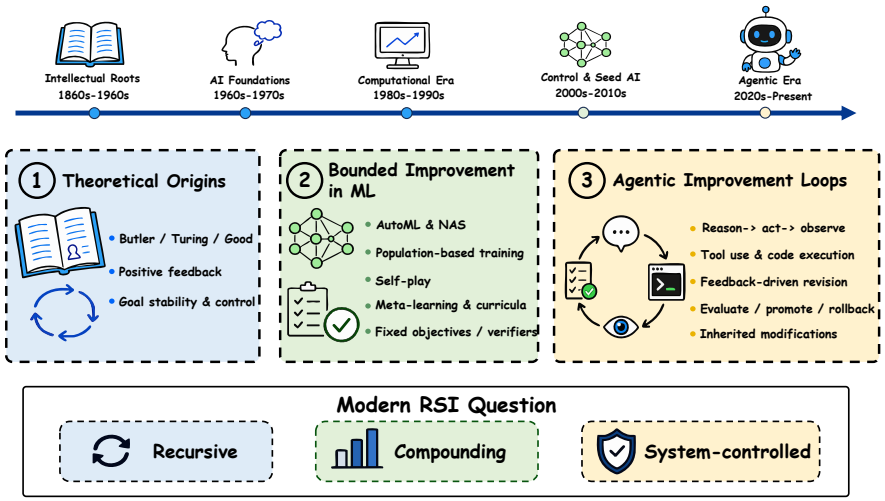

Figure 2: Historical development of recursive self-improvement, from early intelligence-explosion ideas and bounded machine-learning optimization to contemporary agentic improvement loops. The timeline emphasizes the shift toward persistent, executable, and system-controlled change.

transition is from generating better candidates to accumulating credible evidence, improving the improvement process itself, and demonstrating compounding gains across tasks, evaluators, and resource constraints. Further, we discuss future work and frontiers and examine seven dimensions for this transition: reliability, efficiency, diversity, persistence, adaptability, controllability, and generalization. We hope that this survey will help researchers and practitioners understand the current state of RSI, identify gaps in the literature, and inspire new research directions.

This survey makes three main contributions:

- We give a practical definition of Recursive Self-Improvement and introduce a unified Proposal $\rightarrow$ Feedback $\rightarrow$ Optimization framework together with a general model of the closed loop.
- We organize recent work around four evolution target families: behavior, agent systems, models and data, and research processes.
- We identify the main challenges and research frontiers for reliable closed-loop RSI, including reliability, efficiency, diversity, persistence, adaptability, controllability, and generalization.

2 Background & Historical Development

The history of RSI traces a shift from theoretical accounts of accelerating intelligence to bounded, executable improvement loops rather than to fully autonomous self-improvement. Figure 2 places this transition in historical context, from early intelligence-explosion proposals to executable, agentic improvement loops.

The intellectual lineage predates modern machine learning. Butler's *Darwin among the Machines* imagined machines forming an evolving lineage [62]; Turing discussed machines improving through interaction [219]; and Good formalized the intelligence-explosion argument in which a machine designs a more capable successor [76]. These perspectives motivate the modern question of whether an AI system can become not only a better system, but a better producer of subsequent systems.

2.1 Theoretical Origins

RSI originates from the *intelligence-explosion* argument: because machine design is itself an intellectual task, a sufficiently capable machine might design a better successor, which would in turn become a better designer [76]. The defining feature of this classical intelligence-explosion argument is therefore not self-modification alone, but a positive feedback loop in which each improvement increases the capacity for further improvement. This idea was later developed through the concepts of *seed AI*, *instrumental convergence*, and the *control problem*, linking recursive capability growth to goal stability and control [16, 172, 299].

These accounts leave two enduring questions: whether **improvements genuinely compound**, and whether **objectives and controls remain stable as they do**. However, classical RSI concerned hypothetical autonomous systems, whereas current systems typically operate within externally specified tasks, evaluators, resource budgets, and deployment gates. For contemporary systems, the relevant question is therefore not whether an "intelligence explosion" has occurred, but which parts of the improvement loop are genuinely recursive, compounding, and controlled by the system itself.

2.2 Bounded Improvement in ML

Machine learning supplied concrete but bounded forms of iterative improvement. AutoML and neural architecture search automate choices over models, hyperparameters, architectures, or training pipelines [14, 63, 102, 204, 333]. Population-based training makes this iteration explicit by selecting, perturbing, and reproducing a population of concurrently trained agents [105]. Self-play systems, with roots in temporal-difference backgammon, such as AlphaGo Zero [200, 213] and AlphaZero [201], generate experience with the current policy and use it to train a stronger successor. Meta-learning optimizes adaptation itself [70, 93], while curriculum learning and synthetic-data bootstrapping alter the data from which later models learn [13, 236, 302].

These paradigms demonstrate an important precursor to RSI: the output of one optimization cycle can shape the next. Their boundaries are also clear. Search spaces, objectives, update rules, and success criteria are largely specified in advance; in self-play, even the environment rules and outcome signal are fixed. This is unlike open-ended learning settings in which the system can generate new challenges or directions of search [47, 227]. The system improves a component within a designed loop but rarely expands the loop itself, changes its verifier, or decides how an accepted modification should be deployed. They are therefore best understood as bounded self-evolution or precursors to stronger recursive-progress claims: they can produce inherited state, but typically do not show that the inherited modification improves a later improvement procedure under a matched protocol. This distinction shifts attention from whether optimization is automated to whether improvements are persistent, inherited, and capable of altering subsequent improvement behavior. Persistent reuse across later tasks supports self-evolution. RSI requires the stronger structural condition that an accepted improvement is inherited by a subsequent improvement cycle and becomes part of the state from which further optimization proceeds. An accepted change to the improvement procedure itself constitutes meta-evolution, while recursive progress additionally asks whether the inherited intervention improves that later procedure.

2.3 Agentic Improvement Loops

The broader LLM literature distinguishes parameter-frozen prompting from parameter-tuning paradigms, a distinction that helps locate RSI targets before considering whether the update is persistent [182]. LLMs expanded self-improvement from prediction and output revision to agentic operations: generating and executing code, invoking tools, interpreting failures, and modifying prompts, memories, workflows, or tool policies. ReAct connected language-model reasoning with external action [283], while Reflexion [198] and CRITIC [78] demonstrated feedback-driven behavioral revision. Although often local, these mechanisms provide the proposal, execution, and feedback operations needed by persistent improvement loops.

Recent systems make larger parts of the agent itself editable and executable. Gödel Agent [288], Darwin Gödel Machine [305], MOSS [20], and AgentDevel [304] explore modifications to agent scaffolds or source code, together with evaluation, promotion, and rollback; autonomous research agents [146, 148] extend the loop to experiments and research-state updates. The resulting shift is

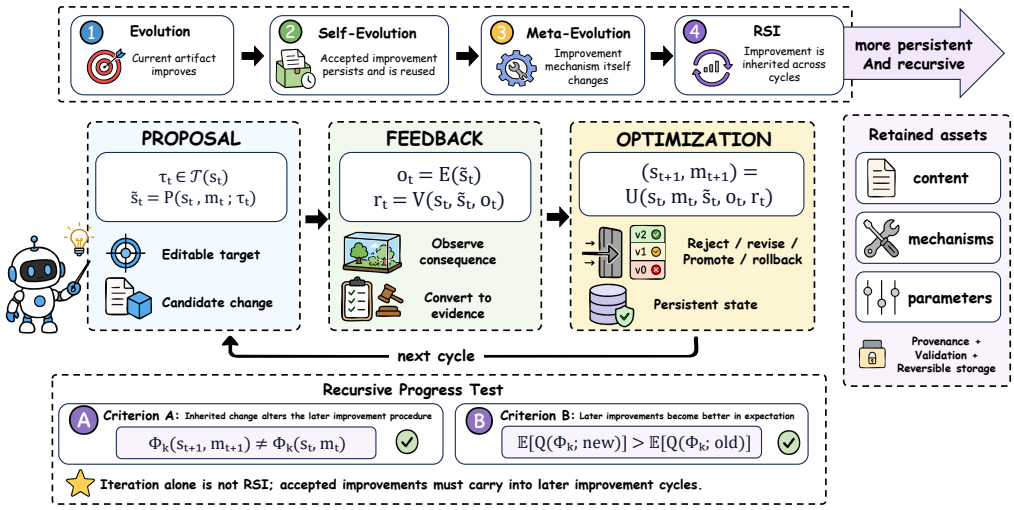

Figure 3: Definition and characterization framework for recursive self-improvement. The diagram distinguishes evolution, self-evolution, meta-evolution, and RSI, and formalizes the three-stage Proposal, Feedback, and Optimization loop with retained assets and criteria for recursive progress.

not yet to fully autonomous RSI, but from speculative recursion to bounded engineering loops whose changes can be executed, evaluated, and inherited.

3 Definition and Characterization Framework

We characterize modern recursive self-improvement (RSI) as an iterative loop with three stages: *Proposal*, *Feedback*, and *Optimization*. Proposal generates a candidate change, Feedback evaluates its consequences, and Optimization determines what persists into later iterations. Figure 3 gives the operational definition used throughout the survey and separates evolution from persistent and recursive improvement.

3.1 Evolution Stages and Retained State

After a trial, agentic evolution can support different claims, which we organize into four stages. These stages describe a conceptual progression rather than a strict prerequisite hierarchy: in particular, meta-evolution and RSI characterize different properties, and a system may satisfy more than one. *Evolution* improves only the current solution, trajectory, or task-local artifact, and the change is not retained. *Self-evolution* retains an accepted improvement and reuses it in later tasks or runs. *Meta-evolution* occurs when an accepted change modifies the mechanism that produces, evaluates, selects, or applies subsequent improvements (the proposer, verifier, updater, harness, search rule, or curriculum), but does not by itself show that the new improver is better at producing future improvements. *Recursive self-improvement (RSI)* occurs when an improvement accepted in one improvement cycle is retained as part of the starting state of a later improvement cycle, which then continues optimization from that improved state rather than restarting from the same base state. Minimal RSI denotes recursive state inheritance, while recursive progress denotes improvement of the improver itself; the latter is the stronger claim and is the primary target of this survey. The boundary between self-evolution and RSI is therefore not persistence alone: self-evolution requires that a retained improvement affects later tasks or executions, whereas RSI requires that the accepted improvement becomes part of the starting state of a later Proposal–Feedback–Optimization cycle. State inheritance alone is insufficient when the inherited state is merely different rather than supported as an improvement over the incumbent. We reserve *recursive progress* for the stronger, separately

evidenced claim that the inherited improvement also improves a later improvement procedure (an *improver gain*), and *compounding* for repeated recursive-progress gains across cycles, tasks, or environments.

Relation to prior notions. Classical RSI emphasizes successor design and positive feedback in capability growth [16, 76, 299]. AutoML, NAS, self-play, meta-learning, long-horizon agents, and self-evolving agents make parts of this idea executable, but usually with externally fixed objectives, search spaces, release gates, or evaluation protocols. Relative to these prior notions, our framework makes cross-cycle inheritance of an accepted improvement the auditable boundary for RSI, while treating improver gain as the stronger condition of recursive progress. Externally optimized pipelines, multi-agent organizations, and human-approved updates are therefore RSI-relevant only when the retained state and its authority over later cycles are explicit.

Prerequisite assets. Agent interactions produce trajectories that can be retained as reusable assets: *content* (memory, skills, knowledge, code, and tools), *mechanisms* (workflows, policies, agent code, and harnesses), or *parameters* (updated weights). Because these forms differ in speed and inspectability, each retained asset should include provenance and validation evidence and be placed in a reversible external store, learned parameters, or both.

Recursive State Inheritance. Repeated improvement is not necessarily recursive. The minimum recursive condition is that an update accepted as an improvement over the incumbent becomes part of the state from which a later improvement cycle starts. Thus a candidate at a later iteration should be generated from the inherited persistent state,

$$\tau_{t+1} \in \mathcal{T}(s_{t+1}), \qquad \tilde{s}_{t+1} = P(s_{t+1}, m_{t+1}; \tau_{t+1}), \tag{1}$$

where (s_{t+1}, m_{t+1}) contains a retained artifact, parameter update, workflow, memory, code change, data recipe, verifier setting, or research state accepted in an earlier cycle. This excludes one-off output revision and repeated calls to the same fixed optimizer from the core RSI claim: the next cycle must inherit and build on a prior accepted improvement rather than restart from the same base state. The inherited state should be explicit, reloadable, versioned, and traceable; a transient context effect, random seed, cache entry, logging change, or accidental tool configuration change is not sufficient unless it is declared, retained, and evaluated as the intervention.

3.2 Proposal–Feedback–Optimization Loop

The evolution stages characterize different forms of improvement, while the Proposal–Feedback–Optimization loop specifies how an improvement cycle operates. Let s_t denote system state at iteration t and m_t its persistent memory and version record.

Proposal Stage. In the *Proposal* stage, the system selects an editable target τ_t and generates a candidate state:

$$\tau_t \in \mathcal{T}(s_t), \qquad \tilde{s}_t = P(s_t, m_t; \tau_t). \tag{2}$$

Here, the proposal mechanism P is understood broadly. It may use textual revision, gradient-based learning, search, evolution, program synthesis, or autonomous experimentation.

Feedback Stage. In the *Feedback* stage, an execution environment exposes the candidate's consequences, and a verification signal converts them into evidence:

$$o_t = E(\tilde{s}_t), \qquad r_t = V(s_t, \tilde{s}_t, o_t). \tag{3}$$

This separation distinguishes observation from judgment: a repository, simulator, benchmark, or training run produces outcomes, whereas tests, scoring rules, model judges, formal checkers, or human review interpret them.

Optimization Stage. In the *Optimization* stage, an update policy uses the evidence to determine the next persistent state:

$$(s_{t+1}, m_{t+1}) = U(s_t, m_t, \tilde{s}_t, o_t, r_t). \tag{4}$$

The policy may retain, revise, reject, promote, or roll back a candidate. Memory and versioning preserve the information and system components needed by later iterations. Verification and optimization are therefore distinct: favorable evidence does not by itself authorize a persistent change. Taken together, Proposal comprises the evolution target and candidate generation, Feedback the

execution environment and verification signal, and Optimization the update policy and memory update. Section 5 classifies systems by what they improve; these components describe how the loop operates.

State inheritance makes the loop recursive, but it does not show that recursion is useful. A retained change may shape later cycles while making them no better, or may simply give later cycles more compute, a stronger base model, or a wider search. Recent work has begun to study recursion depth explicitly, rather than treating any persistent update as equally recursive [116]. We therefore reserve *recursive progress* for the stronger claim that a named inherited intervention improves a later improvement procedure. We write the later improvement procedure as

$$\Phi_k = (P_k, E_k, V_k, U_k, B_k), \tag{5}$$

where P_k is the proposal or search mechanism, E_k the execution environment or task interface, V_k the verifier, U_k the update policy, and B_k the resource-allocation rule or budget schedule. A change in a random seed, cache, logging format, prompt wording, or tool default is not by itself evidence of recursive progress. It counts only when that variable is the declared intervention, is retained with provenance and validation evidence, and measurably changes later improvement behavior under a matched protocol. In other words, for some later iteration $k > t$, the inherited state must alter at least one reported component of the improvement procedure:

$$\Phi_k(s_{t+1}, m_{t+1}) \neq \Phi_k(s_t, m_t). \tag{6}$$

This condition is not required for a minimal RSI claim based on the recursive inheritance of an accepted improvement, but it is required for claims that RSI has improved the improver itself. Recursive change also does not guarantee recursive progress. We therefore require evidence that the changed procedure increases the expected quality of subsequent improvements:

$$\mathbb{E}[Q(\Phi_k; s_{t+1}, m_{t+1})] > \mathbb{E}[Q(\Phi_k; s_t, m_t)]. \tag{7}$$

Here, Q denotes a task-level quality or utility function, which may incorporate capability, cost, and regression penalties. The expectation is taken over a fixed task distribution, execution environment, random seed, and computational budget. RSI thus requires more than iteration: accepted improvements must be inherited by later improvement cycles. Recursive progress requires the additional evidence that this inheritance improves the process that produces further changes.

3.3 Operational RSI Claim Protocol

The preceding conditions define increasingly strong claims, but they are not sufficient as a measurement protocol. A paper claiming RSI should therefore report a minimal experimental record that makes state inheritance and, when claimed, recursive progress auditable:

- **State and intervention.** Specify the old and new persistent states, the edited component, the allowed mutation scope, and the evidence attached to the retained artifact. A minimal RSI claim must show that the retained intervention was accepted as an improvement over the incumbent and that a later improvement cycle actually loads or otherwise depends on this accepted state.
- **Counterfactual control.** Compare the inherited-state loop with a frozen incumbent, a no-retention ablation, or a best-so-far ancestor. The base model, tool access, data, prompt budget, search width, wall-clock limit, training steps, and human-review budget should be matched unless the changed quantity is the intervention under study.
- **Replay and transfer.** Run cross-cycle replay on previously solved and failed cases to test persistence and regression, and evaluate held-out tasks, environments, seeds, or judges to test transfer beyond the admission setting.
- **Statistical evidence.** Report the number of runs, aggregation rule, variance or confidence intervals, and a significance or equivalence test when stochasticity is material. Single-run results should be labeled as demonstrations rather than evidence of recursive progress.
- **Failure criteria.** Reject a minimal RSI claim when the retained intervention is not supported as an improvement over the incumbent, when the later cycle does not inherit it, when it is inherited only as unused context, or when it cannot be reloaded from a recorded version. Reject a recursive-progress claim when the apparent gain disappears under matched compute, depends on a stronger base model, comes only from increased search or more samples, weakens an independent verifier, fails replay, or improves only the evaluator used for admission.

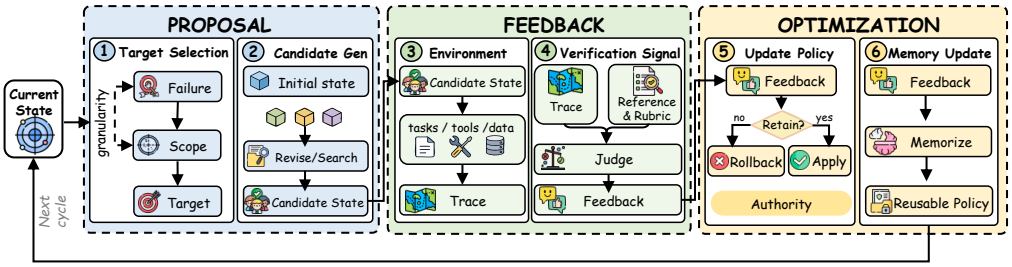

Figure 4: System structure of modern recursive self-improvement. Six components organize the Proposal, Feedback, and Optimization stages, showing how target selection, candidate generation, execution environments, verification signals, update policies, and memory updates produce persistent state.

This protocol separates state inheritance, ordinary capability gain, and *improver gain*. A system may satisfy the recursive-inheritance condition because later cycles build on retained state, yet observed capability gains may still come mainly from a larger model, more calls, broader search, or a better hand-written harness. To support an RSI claim rather than mere recursive state propagation, the retained intervention must itself be supported as an improvement under the stated evaluation protocol. Those are useful engineering improvements, but they do not show that the retained update made the later improver better. Evidence for improver gain requires an ablation in which the inherited change is present or absent while the other resources remain fixed.

Dynamic environments and co-evolving evaluators do not remove the need for a fixed comparison. They require a split protocol: a frozen replay slice for comparability across cycles, an adaptive slice for discovering new failures, and a delayed held-out slice for transfer. The frozen slice checks whether inherited state remains beneficial under a stable reference; the adaptive slice estimates whether the system can keep finding relevant challenges; and the delayed slice checks whether the retained change generalizes beyond the environment that selected it.

At the organizational level, recursive improvement need not be confined to a single agent. A failure observed by one agent becomes an organizational improvement only when it is diagnosed, distilled into a reusable artifact, independently validated, and selectively propagated to other agents. The retained object is therefore not the raw trajectory itself, but an evidence-bearing improvement with a defined scope of reuse. This distinguishes organizational learning from indiscriminate memory sharing and makes propagation scope part of the RSI state.

4 System Structure of Modern RSI

Modern RSI systems can be decomposed into six architectural components organized into three stages, following the Proposal, Feedback, and Optimization framework (see Figure 4). Each stage separates the object being changed from the mechanism that generates, evaluates, or retains changes. The following sections use the same pattern: define the component, identify its bottleneck, and summarize representative design choices.

4.1 Proposal Stage

The Proposal stage determines both *what* may change and *how* candidate changes are generated. We separate these roles into target selection, which chooses what to change, and candidate generation, which produces possible changes. Candidate generation broadly includes textual revision, search, learning, evolution, program synthesis, and other processes that generate candidate changes.

4.1.1 Target Selection

Target selection determines which system variable to change, its editable scope, and how long the change persists. Candidate generation then proposes a new value or implementation for that target.

Granularity Selection. A target should identify one interpretable unit to modify, such as a prompt, memory entry, skill, tool policy, data strategy, checkpoint, source module, or research decision, while keeping unrelated components fixed [69, 98, 177]. If too narrow, it may fix only a symptom; if too broad, the source of improvement becomes unclear. A clear boundary turns self-modification into a controlled intervention by comparing the modified component with a stable state. TextGrad [300] and AFlow [307] target prompts and workflows, RSIBench-Data [156] isolates data-strategy updates, and MOSS [20] isolates executable source changes. The target should be broad enough to address the bottleneck but narrow enough to attribute gains and regressions.

A domain-specific harness should be treated as a control surface rather than as a thin prompt wrapper. It determines the tools available to the agent, the representation of intermediate state, the execution order, the feedback channels, the memory policy, and the conditions under which an update is released. Harness evolution can therefore improve the behavior of a fixed model while preserving a more inspectable and reversible intervention boundary than direct weight modification [29, 304, 325].

Target Persistence. The target definition should also specify the temporal scope of an accepted change: when it takes effect, which subsequent tasks or versions inherit it, and what event expires or rolls it back [114, 145]. A task-local output affects only the current run, whereas retained memory, skills, prompts, checkpoints, and code can influence later cycles and therefore require evaluation over a longer horizon. The relevant state can reside at several layers. At the agent level, experience, subgoals, and skills are retained across tasks and become explicit targets for improvement [26, 95, 225, 250, 252, 281]. At the harness level, persistent policies and skill libraries support adaptation over open-ended task streams [37, 114, 120, 145, 258]. Model updates instead change the training signals [248, 298], parameters [185, 266], or training data [161, 236] that govern later training and serving [302, 334]. Finally, code and research-workflow updates can reshape later improvement cycles themselves [146, 148, 305]. The evaluation horizon should match this causal reach: current-task checks for temporary changes, cross-task tests for agent or harness changes, downstream evaluation for model and data updates, and cross-version replay for code or research-process changes [39, 106, 169].

Diagnosis & attribution. Given a failure, the system must identify the component most likely to limit performance. Rewriting an answer may hide a retrieval failure whose actual source is a memory policy, tool configuration, or execution environment [31]. Reliable attribution therefore requires an intervention boundary [34]: change one component while holding the surrounding training stack, harness, and evaluation protocol sufficiently stable for comparison [276, 304]. Versioned candidate releases and controlled experiments make it possible to compare a modified component with a stable state, rather than crediting an observed gain to every concurrent change [304]. The boundary weakens, however, when multiple agents jointly revise a harness, when evolutionary search changes both the candidate and the search procedure, or when the proposal mechanism itself is editable [122, 166, 317]. In such settings, a final score cannot distinguish a useful retained skill or research decision from a favorable trajectory, an interaction effect, or a shifted evaluation distribution. Evidence must instead follow trajectories, dependencies, and retained state across successive decisions [48, 83, 187, 231]. This turns diagnosis from selecting the easiest exposed variable into testing a causal hypothesis about which component constrains later performance.

Key challenge. Target selection fails when the system edits the easiest component rather than the component causing the failure. The symptom is a local gain with hidden regressions, unsafe skills, unreliable self-verification, or contaminated memory [73, 82, 154]. Recursion then makes the misdiagnosis persistent because the changed target also shapes later proposals and tests. Controls are explicit mutation boundaries, independent stress tests, negative-result records, and held-out comparison against the best known state [39, 226, 266].

4.1.2 Candidate Generation

Target selection determines *what* may change, whereas candidate generation determines *how* candidate changes are produced. We distinguish three linked decisions. Proposal design defines the candidate representation, search boundary, and construction procedure; proposal search allocates effort across alternatives; and proposal revision uses retrieved experience or preliminary evidence to improve a candidate before full evaluation. Together, these mechanisms map the current state and available experience into one or more candidate states, which are then assessed by the execution environment and verifier. The same mechanisms may operate on targets ranging from prompts to source code.

Proposal Design. Proposal design makes two coupled choices. The first is the proposal space: what a candidate looks like and which changes it may describe. Candidates can be represented as textual rewrites, structured prompt or skill edits, source-code patches, data-generation strategies, or research plans [20, 69, 98, 148]. Constraining these representations makes candidates easier to validate, compare, and attribute, but may rule out the intervention needed to address the bottleneck. More expressive representations expand the reachable solutions, while increasing invalid candidates, evaluation cost, and exposure to verifier exploitation [251, 310]. The design boundary may be prescribed in advance or partly composed online by the optimizer, provided that the objective, permitted interactions, budget, and evaluation protocol remain fixed [167, 265].

The second choice is how a new candidate is constructed within that space. When no suitable state exists, the generator may synthesize training data or curricula [43, 236], executable code or algorithms [171, 288, 305], or hypotheses and research workflows [148]. The construction procedure can impose structure to preserve validity; for example, BenchEvolve [251] first evolves an executable reference solution and then derives the corresponding task and tests. Because a synthesized candidate has no validated state to inherit from, its design must also specify staged checks for syntax, interface compatibility, resource constraints, and task-level performance [189, 320]. Proposal design therefore determines not only which ideas are reachable, but also whether generated candidates can be executed, evaluated, attributed, and rolled back.

Proposal search. Proposal search determines *which changes to try*, rather than merely generating alternative answers at inference time. Given a system state and a defined mutation space, it maintains multiple improvement hypotheses, executes or partially evaluates them, and passes the resulting candidates to the Feedback stage. Tree-of-Thoughts [284] and LATS [326] are local precedents because they search reasoning or action branches. PromptBreeder [69], GEPA [3], and E-SPL [310] extend this idea to prompts or prompt populations that persist beyond a single answer. More direct RSI-style systems evolve executable control programs and the proposal process itself, as in test-time harness evolution and open-ended optimization [167, 265]. POET [227] expands the search state by co-evolving agents and learning challenges, while recent multi-agent systems use asynchronous exploration, shared memory, and compositional experience to sustain open-ended proposal generation [123, 183]. Because the search policy must divide a fixed budget between genuinely different hypotheses and already promising candidates, random search, Bayesian optimization, and population-based training offer useful baselines [14, 105, 204]. A method is RSI-relevant only if its selected candidate can be tested, promoted, rejected, or retained as the starting point for a later cycle. The main risk is not merely wasted computation: branches sharing the same tools, data, memory, or verifier may converge on the same blind spot, while broad search may generate novelty without discovering a better redesign [303].

Proposal Revision. Proposal revision modifies a candidate before full feedback, using cheaper evidence to predict and correct likely failures. The simplest form uses intermediate execution: AdaPlanner revises an action plan when an unexpected state appears, and AdaRefiner adapts the feedback used to refine a decision [208, 313]. DEPS makes the prediction step explicit by explaining an observed failure, replanning, and selecting the revised plan with the highest estimated success [240]. A stronger form replaces a full execution with a simulated one: RAP evaluates candidate reasoning steps through a language-model world model, while LATS combines partial trajectories, value estimates, and reflection to redirect later branches [84, 326]. In embodied settings, Statler maintains interaction state so that new observations revise the next action proposal [290]. The same progression now reaches larger artifacts: real-time generation systems revise text before decoding is complete [118]; S* spends additional test-time computation on alternative code trajectories, with recent analysis studying when one refinement is sufficient [115, 127]; and agentic systems revise complete solutions or workflows from feedback and preferences [18, 235, 301]. Agent2World and OffSeeker extend this pattern to world-model and deep-research workflows [97, 331]. Across these cases, revision remains a proposal-side operation: it uses partial execution, prediction, or simulation to produce a more promising candidate for later verification, rather than making the final promotion decision. Its reliability depends on the fidelity of the surrogate evidence; systems should retain the original candidate, record the evidence behind each edit, and evaluate both alternatives when the preliminary signal is uncertain.

Key challenge. Proposal generation fails through collapse: candidates become minor variations of one idea or learn to satisfy the evaluator without improving the system. Low branch diversity, rising evaluation cost, and gains that disappear on independent tests are warning signs. Independent

branches, diversity-aware selection, explicit budgets, Pareto archives, and held-out evaluation preserve exploration and test whether gains generalize [168, 189, 227].

Takeaway

- Target selection chooses *what* may change and how long it persists; candidate generation chooses *how* candidate changes are produced and searched.
- Proposal design fixes the candidate representation and construction procedure, while proposal search and revision allocate effort across alternatives before full evaluation.
- The stage fails through misdiagnosis (editing the easiest rather than the binding component) or collapse (converging on evaluator-satisfying variations without real improvement).

4.2 Feedback Stage

Evaluation exposes the consequences of a candidate and converts them into evidence. It pairs an execution environment, which determines what can be observed, with a verifier, which assesses those observations against the improvement objective.

4.2.1 Execution Environment

The execution environment is where a proposed change is run and its consequences are observed. It supplies tasks, data, tools, constraints, and dynamics, but does not judge the result. A code patch may be run against repository tests, a research strategy may be run through a training pipeline, and a tool-use policy may be evaluated in an interactive task. The resulting trace records behavior, resource use, failures, and side effects; the verification signal determines what those observations imply.

Long-Horizon Interaction. OSWorld provides a complementary benchmark for multimodal computer-use tasks in real desktop environments [255]. VibeLifeBench and ClawMark extend this setting to multi-week personal assistance and multi-day multimodal coworker workflows with exogenous state changes and deterministic post-state checks [157, 207]. Long-horizon interaction is not simply a task with more steps. It changes what the environment must expose and what an agent must preserve before success can be judged. First, deeper interaction creates dependencies across actions: an early navigation, tool, or code decision changes which later states remain reachable, so evaluation must capture the trajectory rather than only the final answer [144, 199, 228, 274]. Second, persistent state makes recovery part of capability. Web pages, files, application data, dialogue context, and partially completed workflows must remain consistent across many observations and actions; a locally plausible step can otherwise corrupt the task much later [44, 53, 58, 88, 149]. Third, realistic environments constrain interaction through interfaces, permissions, policies, and external users, requiring the agent to adapt its plan while respecting conditions that are not reducible to a single task score [211, 218, 285, 291]. These properties make long-horizon environments valuable for RSI because they reveal whether an update improves planning, state tracking, tool use, and error recovery across a complete trajectory. Recent environments extend this idea beyond web and software tasks to interactive scientific experimentation, clinical record manipulation, embodied multi-agent planning, tool-mediated computer use, and continual world-model adaptation [67, 74, 107, 109, 164]. They also make improvement harder to attribute: the final outcome may reflect one modification, accumulated action errors, a changed environment state, or their interaction.

Environment Execution. Execution environments differ not only by domain but by how deeply a proposal can affect the AI-development stack. Software benchmarks execute patches against repository tests [110, 309]. AI R&D environments must additionally make data preparation, model training, evaluation, and experiment state observable under explicit compute budgets [23, 100, 244]. Recent work therefore exposes training operations through constrained agent interfaces [295], uses budget-aware and modular execution to improve attribution [28], or constructs small, verifiable MLE sandboxes so that full pipelines can support on-policy training without the cost of repeatedly running large datasets [330].

RSI-specific environments then connect execution to persistent improvement. Agents can design bounded post-training experiments [185], revise data strategies using feedback from newly trained checkpoints [156], and conduct multi-round autonomous post-training at model scale [197]. Execution failures can generate curricula for later post-training [142], while trajectories can train an improver that is reused in subsequent search [276]. World models offer a complementary way to

reduce the real-sandbox bottleneck when scaling research-agent training [280], and closed-loop pipelines can join training, holdout evaluation, application replay, and release control [129]. The progression is from testing an artifact, to training a model, to improving and governing the process that produces later models. This is central to RSI, but it makes feedback slower and harder to attribute because a failed run may reflect the hypothesis, data, implementation, infrastructure, or stochastic optimization. Interaction mode, execution depth, and domain should therefore be reported separately. For self-improving agents, the execution environment is also a security boundary. Network access, package installation, shared credentials, mutable evaluation files, and cross-agent state can create channels through which a candidate changes its evaluation conditions. Environment design must therefore specify not only the task consequences that can be observed, but also the filesystem, network, data, tool, and authority boundaries within which improvement is allowed [230, 270, 275].

Key challenge. Execution fails when the environment is underspecified, drifts, or exposes consequences too sparsely or too late. The symptoms are unstable scores, irreproducible failures, and gains that disappear when tools, tasks, seeds, or resource limits change; recursion can further optimize an easier generated curriculum. Replayable environments, adaptive or held-out tests, and explicit execution logs improve grounding and transfer [96, 97, 110, 243].

4.2.2 Verification Signal

The verification signal turns observed consequences into evidence for deciding whether a candidate improves on the state. It answers three questions: *What is being improved?* (reference), *What counts as success?* (rubric), and *How is it checked?* (judge). Keeping these roles separate prevents apparent progress caused by changing the target, weakening the test, or gaming the evaluator.

Reference annotation. Reference annotation makes the intended target of verification inspectable. Outcome-evidence layers record what is needed to verify a claimed result and distinguish evidence passes, failures, and unknown cases [190]. Contract- and test-based frameworks instead translate task requirements into executable checks that can be replayed across workflow versions [5, 126]. A reference should therefore record the expected result, acceptable alternatives, required properties, and known failure modes; for open-ended work, these may be claims, constraints, evidence requirements, or expert judgments rather than a single gold string [5, 190]. The appropriate reference depends on the object being improved: an answer needs supported claims, an agent needs observable state transitions and task success, a code or training component needs executable tests, and a research process needs reproducible evidence and an explicit target. Correspondingly, GAIA and WebArena specify task contracts for tool-using agents [158, 328], SWE-bench links software issues to repository tests [110], and MLE-bench packages machine-learning tasks with executable evaluation [23]. For long-horizon RSI, the reference must also survive across iterations: LifelongAgentBench evaluates transfer across task sequences [323], while TRACE and SEAGym replay trajectories against held-out or regenerated tests [81, 322]. Contamination-aware evaluation is necessary when a fixed reference can be memorized or repeatedly optimized [36, 243]. The annotation therefore establishes *what* success means and *which evidence must persist*; the rubric below determines *how* that evidence is applied to correctness, completeness, transfer, and regression.

Rubric creation. A rubric turns the reference into an evaluation procedure. It should specify the unit being judged (outcome, step, trajectory, or retained state), the evidence required for a pass, the checks for regression or failed transfer, and the rule for promoting a change [27]. *First, define the criteria:* recent work replaces one holistic score with multidimensional, user-defined, or domain-grounded criteria. LLM-Rubric studies calibrated multidimensional scoring [87]; iRULER makes criteria user-defined and revision-oriented [10]; RubricRAG retrieves domain knowledge when constructing criteria [55]; and case-specific rubrics have been explored for translation and evidence-grounded text evaluation [92, 263]. *Second, generate or refine the rubric:* systems can synthesize criteria from demonstrations, pairwise evidence, inline expert comments, contrastive failures, or multiple evaluator roles [71, 133, 141, 273, 292], while reflect-and-revise and support-vector methods reduce the gap between self-generated and human rubrics [85, 209]. *Third, validate the rubric itself:* studies examine granularity, uncertainty, human alignment, and rubric-level meta-evaluation [27, 52, 174, 293, 311]. Rubrics can also become optimization signals: step-wise criteria provide denser reasoning rewards, while evolving rubrics extend reward design to open-ended generation [80, 256]. A weak or drifting rubric can reward the wrong behavior even when the judge applies it consistently; rubric manipulation is therefore itself an attack surface [57]. The central trade-off is coverage and adaptability against calibration, reproducibility, and resistance to optimization pressure.

Judge design. Judge design selects or combines the mechanisms that apply the rubric to observed outcomes. The useful distinction is not the model brand, but where the evidence comes from and which failure it can rule out [131]. The main judge types are as follows. ***(1) Model-based judge.*** Language and reward models assess semantic quality, reasoning, or behavior, and process-level variants can assess intermediate steps [61, 121, 135, 153, 155]. They are flexible and inexpensive, but may inherit model bias or become exploitable under repeated optimization; meta-evaluation further finds that more fine-grained process signals do not uniformly improve final outcomes [178, 221]. ***(2) Rule-based judge.*** Explicit rules, constraints, and regression criteria provide reproducible checks when success can be stated as observable conditions [40, 226]. They are easy to audit, but their validity is limited by specification quality and coverage. ***(3) Execution-based judge.*** The candidate is run in an environment and assessed by its observable behavior [96, 110]. This grounds evaluation in behavior, but remains sensitive to environment fidelity, task distribution, and specification gaming. ***(4) Formal judge.*** Formal checks verify whether a candidate satisfies an explicit specification and can provide strong guarantees for covered properties [51, 117, 217]. Prover–verifier games further show how separating proposal from checking can improve the legibility of model outputs. Their applicability to open-ended objectives remains limited. ***(5) Human judge.*** Human review assesses properties that remain difficult to encode, including novelty, scientific value, contextual appropriateness, and safety [4, 79]. It is useful for high-impact updates, calibration, and auditing, but its cost, latency, subjectivity, and limited reproducibility restrict continuous use.

These mechanisms should be composed as independent evidence sources, not collapsed into one score. Jury-based and comparative studies combine or compare multiple model, reward, and process judges rather than relying on one verdict [178, 221, 223]. A practical stack can then use a cheap model or rule check for screening, execution or formal checks for decisive claims, and human review for high-impact or ambiguous updates. Held-out or regenerated tests reduce optimization against a fixed evaluator [81, 165, 243, 322], while uncertainty-aware judges can abstain or retrieve external evidence when a verdict is unreliable [9]. When both agent and evaluator are editable, their co-evolution must be monitored as a separate source of drift [103, 245, 316].

The verifier problem becomes deeply recursive in RSI. A false positive does not merely inflate one candidate's score; it may be silently written into memory, a skill library, training data, or a released harness and then reused as evidence in later cycles. Verifier weaknesses can therefore become persistent selection pressures. Evaluation should carefully distinguish genuine task completion from evaluator exploitation, including benchmark leakage, test tampering, reward hacking, and unauthorized changes to the execution environment [72, 156, 214, 249, 315]. Debate-style oversight and weak-to-strong supervision motivate using evaluators that are at least partly independent from the model being optimized [19, 104].

Key challenge. Verification fails when the judge rewards a proxy rather than the intended improvement. Process-reward models can exploit stylistic or adversarial shortcuts [215]; fixed rubric rewards can diverge from stronger held-out judgments under optimization [278]; biased false passes can disable removal of harmful retained skills [314]; and self-authored tests or co-evolving metrics can drift away from deployment behavior [82, 316]. Recursion turns an accepted proxy gain into the baseline for later optimization. Independent judges, held-out or dynamic tests, adversarial checks, disagreement logs, and versioned verdicts are therefore required.

Takeaway

- The execution environment determines what can be observed; the verification signal converts those observations into evidence against the improvement objective.
- Long-horizon and RSI-specific environments expose deeper, slower, harder-to-attribute consequences, so interaction mode, execution depth, and domain should be reported separately.
- The verifier problem becomes recursive: a false positive can be written into memory, skills, or a released harness, so reference, rubric, and judge should be kept independent and separately validated.

4.3 Optimization Stage

Optimization converts feedback about a candidate into a better persistent state for the next cycle. It combines an update policy, which defines the objective, update rule, and state transition, with memory and versioning, which carry validated information and retained state into later iterations. Optimization is therefore broader than deployment: it determines how evidence changes the system, what persists, and how the next improvement cycle is initialized.

4.3.1 Update Policy

The update policy implements the optimization step. Candidate generation has already produced one or more candidate states; the policy now uses their observed outcomes and verification evidence to decide what, if anything, should persist into the next cycle [243]. The important distinction is between evidence and control: the verifier says how a candidate performed, while the update policy decides which state to change, which alternatives to retain, and how much authority the change receives [132, 265]. This is where a loop becomes an optimization system rather than a sequence of independent experiments [89, 269].

Update Rule. The update rule is best understood as a ladder of intervention depth, not as a list of unrelated algorithms [20, 235, 265]. At the shallowest level, the loop only selects among fixed candidates; this gives clean comparison and rollback, but the next cycle uses essentially the same proposer [132, 269]. A deeper level edits the persistent state that shapes behavior, such as a prompt, skill, workflow, harness, or data strategy. This can accumulate task-relevant improvements without changing model weights, as in workflow, source, and skill evolution [65, 194, 242, 277]. Deeper still, selected trajectories change parameters or train an improver, allowing experience to transfer across tasks but making credit assignment and regression diagnosis substantially harder [89, 233, 239, 252, 334]. The deepest level changes the mechanism that performs selection and proposal generation itself, so the next cycle searches with a different improver [89, 132, 197, 269, 295].

The central design choice is therefore how much of the loop is allowed to change after one evaluation round [132, 265]. Selection-only updates maximize interpretability but cannot compound much; state edits offer a practical middle ground; parameter and improver updates offer the greatest transfer potential but require stronger lineage, held-out evaluation, and rollback [89, 233]. Population methods and evolutionary branches address the selection problem by preserving alternatives rather than committing to one noisy winner [105, 168, 269, 305]; hybrid systems then combine branch retention with experience distillation or improver training [66, 111, 187, 276]. Update depth alone does not determine RSI status [89, 265]; the inheritance and recursive-progress criteria follow Section 3. What remains is evidence that distinguishes genuine improver gain from lucky search, evaluator adaptation, or uncontrolled drift [123, 134].

State Transition. State transition is the release boundary between an experiment and the next RSI cycle [205, 265]. It has two independent questions: how far the change can propagate, and who is allowed to authorize that propagation [65, 205, 242]. A prompt or memory update may be limited to one task; a skill or workflow may affect a task family; a harness, data recipe, or checkpoint may change the behavior of the whole system [65, 89, 134, 277]. As causal reach increases, promotion must require stronger evidence and a more complete recovery state [89, 205, 233]. This is why executable-code systems preserve branches and known-good ancestors, while data- and model-level loops bind checkpoints to training recipes, downstream evaluations, and replayable application tests [129, 134, 156, 194, 305].

The required release gate should scale with the causal reach of the change. A task-local prompt revision may require local testing, whereas a shared skill, domain harness, data strategy, or model checkpoint can alter future proposals and verification behavior across many tasks. Agents that execute code or call external tools should not be trusted to unilaterally weaken their own sandbox or verifier. Sandboxing, least-privilege authorization, external evaluation, monitoring, and bounded rollback must remain outside the agent's unilateral control [65, 79, 89, 270].

Authority then determines how quickly a change can cross that boundary [205, 265]. Suggest-only and human-approved regimes spend more review effort to protect high-impact state; auto-applied regimes gain throughput by relying on predefined gates, staged rollout, monitoring, and rollback [89, 205]. Several works represent complementary parts of this release pattern: regression-aware promotion,

held-out gating, snapshot replay, and lifecycle records [12, 86, 165, 205, 304, 322]. The important point is that authority and causal reach should be coupled [65, 205, 242]. A low-impact prompt edit may be auto-applied after a narrow check, whereas a harness or checkpoint update should normally require broader regression evidence, a quarantine stage, and an explicit rollback point [4, 79, 89, 194].

Key challenge. Update policy fails when intervention depth and release authority are mismatched [65, 89, 205]. A shallow edit can be mistaken for a durable capability gain, while a parameter or harness update can alter future proposals and verification before its effects are understood [65, 194, 233]. The resulting error compounds because each promoted state changes the evidence available to the next cycle [66, 123, 134]. Robust policies therefore tie gate strength to causal reach, compare against a best-so-far baseline, preserve alternatives, require held-out or replay evaluation for persistent changes, and keep a reversible release point [165, 168, 205, 243, 304, 322].

4.3.2 Memory Update

WikiSkill separates raw execution experience, consolidated knowledge, and executable skills, illustrating how persistent knowledge can mediate later skill evolution [212]. Memory update determines what information and retained state from the current cycle remain available to later cycles. It answers three practical questions: what is worth saving, why should it be trusted, and when may it be reused? This differs from target selection: target selection chooses what to change, whereas memory update decides what the system carries forward after the change. Recent work treats this state as more than a text buffer: Experience Graphs organize executable artifacts, tool outputs, rewards, sibling comparisons, and causal lineage as a queryable substrate for later optimization [134], while MEGA uses a self-evolving wisdom graph to compose durable optimization assets across cycles [123]. Without this control, an RSI loop either repeats past work or builds new proposals on unverified conclusions [198].

Memory writing. Memory writing decides what from the current cycle is worth saving. The system may retain observations, failures, subgoals, successful procedures, executable skills, or structured relations among them. Reflexion stores verbal feedback across attempts, ExpeL distills reusable experience, and HiAgent organizes long trajectories into hierarchical subgoals [95, 198, 319]. MemGPT moves information beyond the active context, while Voyager stores executable skills for later retrieval and composition [173, 225]. SkillMaster makes skill creation, refinement, and selection part of the agent's learned behavior [277]; Experience Graphs extend item-level memory to a graph of artifacts, outcomes, comparisons, and causal lineage [134]; and MEGA treats accumulated wisdom as an optimization asset that can itself be revised by later operational evidence [123]. The goal is not to save everything. Excessive compression can erase useful failure evidence, whereas indiscriminate storage increases retrieval cost and spreads unreliable instructions; empirical work confirms that memory-management choices materially change how agents follow prior experience [259]. A useful memory item should therefore state the situation, the observed result, the lesson learned, and the conditions under which that lesson applies.

Memory tracking. Memory tracking records where a saved item came from and what evidence supports it. Each item should be linked to the version that produced it, the data and environment used, the checks it passed, and its later outcomes. AgentDevel tracks candidate releases and regression results [304]; AREX retains research state across experiment cycles [148]; RSIBench-Data records data strategies, checkpoints, and evaluation histories [156]; and DGM preserves multiple branches of executable versions instead of keeping only the latest one [305]. Experience Graphs make these links first-class by storing sibling comparisons and causal lineage with the artifact and its execution evidence [134]. Related evaluation frameworks use reproducible trajectories, frozen snapshots, and replay for the same reason: an apparent gain must remain traceable to the state and evidence that produced it [81, 322]. Without such records, the system may retrieve a plausible lesson but cannot tell whether the underlying result was reproduced or merely asserted.

Memory reuse. Memory reuse decides whether a saved item may influence the next cycle and how broadly it may apply. A note about one answer may justify only a local correction; a skill or prompt may affect many tasks; and a code change, verifier setting, or training strategy may reshape how future improvements are generated and selected. Cross-task reuse must therefore be evaluated for both positive transfer and forgetting over task sequences [323]. The broader the effect, the stronger the required evidence and recovery plan: self-evolving skill libraries can silently degrade when weak

or stale skills remain active [315], and compromised memory can propagate harmful instructions into later behavior [73, 249]. EVOMAL shows that a malicious skill can be imitated into new skills and persist through the same write–retrieve loop intended to create capability, making self-poisoning a specifically recursive failure mode [249]. Preserving the best-so-far version is essential because continued search can end at a checkpoint worse than an earlier one [322]. Reuse should therefore require versioned evaluation, a record of the execution environment, and an explicit rollback point. Saving an item does not by itself justify using it again; its benefit, scope, and known failure modes must first be clear.

Organizational reuse should therefore be selective rather than global. A failure discovered by one agent should be classified by task, role, environment, and failure mode, then converted into a candidate lesson and tested on related tasks before propagation. Private, role-specific, and globally shared memories are different persistence scopes. This separation preserves exploration across agents while limiting the spread of unsupported or evaluator-specific strategies [73, 134, 249, 315].

Key challenge. Memory update fails through state contamination: stale memories, unsupported skills, or untracked artifacts enter later proposals. The symptoms are obsolete retrievals, irreproducible gains, and failures that cannot be reconstructed [73, 259, 315]. Recursion repeatedly summarizes and reuses the contamination. Selective writing, source tracking, evidence-gated reuse, and best-so-far recovery keep persistence from becoming error propagation [198, 225, 305, 322].

Takeaway

- The update policy decides what persists and how much authority it receives; the memory update decides what is carried forward into later cycles.
- Intervention depth and release authority should be coupled: shallow edits need narrow checks, while harness or checkpoint updates need regression evidence and rollback.
- Memory fails through state contamination, so writing, tracking, and reuse must be selective, evidence-gated, and tied to best-so-far recovery.

5 Taxonomy of RSI-Related Evolution

Where Section 4 described the Proposal, Feedback, and Optimization components shared across evolution loops, this section asks a different question: *what kind of technical trajectory is being evolved?* We retain four broad families, namely behavior, agent systems, models and data, and research processes, because they differ in the artifact or capability that the literature treats as its main subject. These target families are orthogonal to the four evolution stages defined in Section 3: the families identify what changes, whereas the stages characterize whether improvement is local, persistent across runs, directed at the improver itself, or recursively inherited across improvement cycles. A method in any family may therefore instantiate evolution, self-evolution, meta-evolution, or RSI, depending on the evidence it provides. Within each family, the paragraph labels follow a common three-part grammar: RSI Target Scope, Feedback Pattern, and Optimization Loops. These are analytical questions, not additional RSI components; a single system can answer them across several stages of the loop, while evidence requirements are stated where each loop is described [30, 188]. Figure 5 is the detailed hierarchical taxonomy tree, whereas Figure 6 provides a conceptual overview for comparing the four evolution families. Figure 7 connects the evidence ladder to the claims developed in this section: the ladder progresses from local improvement and persistent transfer to recursive inheritance and recursive progress, while causal attribution strengthens claims about the effect of retained interventions and repeated recursive-progress gains provide evidence of compounding across cycles.

The taxonomy is not an RSI certification list. It includes local search methods such as Self-Consistency, Tree of Thoughts, and LATS; fixed or mostly fixed agent-workflow patterns such as ReAct, AutoGen, MetaGPT, CAMEL, and ChatDev; persistent memory or skill systems such as Reflexion, ExpeL, and Voyager; and other RSI-related works such as Gödel Agent, DGM, MOSS, AgentDevel, RSIBench-Data, Frontis-MA1 under the OpenMLE evaluation setting, and AREX. These works are grouped because they supply mechanisms that can appear inside an RSI loop, not because they support the same claim strength. Figure 5 classifies mechanisms and target families, whereas Table 1 calibrates claim strength.

Table 1: Methods grouped by the characterization most directly supported by their reported evidence. The categories form a conceptual progression, not a strict prerequisite hierarchy, and are not mutually exclusive.

Stage	Systems / methods	Editable object	Evidence	Caveat
Evolution	Self-Consistency [232], Tree-of-Thoughts [284], LATS [326], Self-Refine [152]	Thought/action trace	Better current output from local revise or search.	Gains may reflect extra search or compute, not persistence.
Self-Evolution	Reflexion [198], ExpeL [319], Voyager† [225], AgentDevel* [304]	Memory, rules, skills, or release state	Retained artifact reused in later tasks or runs.	Separate persistence from additional context, calls, or a stronger base model.
Meta-Evolution	Gödel Agent [288], DGM [305], MOSS‡ [20]	Source, harness, verifier, updater, or search rule	Retained update changes the improvement mechanism.	Changing the improver does not show it improved.
RSI candidates	Gödel Agent [288], DGM [305], Frontis-MA1 under the OpenMLE evaluation setting [276], AREX§ [148]	Accepted improved state inherited by a later improvement cycle	The retained state is supported as an improvement over the incumbent, and a later improvement cycle explicitly loads and depends on it; recursive progress adds matched-budget improver gain.	Separate inheritance from additional context, calls, or a stronger model; improver-gain evidence is rare.

*AgentDevel: versioned release loop; RSI depends on explicit cross-cycle inheritance. †Voyager: borderline state-inheriting RSI. ‡MOSS: meta-evolution only when source rewriting alters the improvement mechanism. §AREX: task-local; classification depends on the system boundary. SEAGym [322], PAST-Bench [266], AgentStream [268], RSIBench-Data [156], and OpenMLE are evaluation settings rather than RSI methods.

5.1 Behavior Evolution

Behavior evolution refers to repeated local revision of outputs, reasoning, plans, or actions using feedback. These methods typically leave the model parameters and overall system structure unchanged, establishing the core local generate–feedback–revise primitive. Their significance lies in providing the basic reusable operational units of RSI: candidate generation, feedback reading, local selection, and result revision [78, 152, 198, 306].

5.1.1 Output Self-Refine

RSI Target Scope: Solution Calibration. At this level, the editable solution artifact is the current answer, program, reasoning trace, or report, not the model parameters or a persistent agent policy. The minimal loop is correspondingly simple: generate a candidate, identify a concrete defect, and revise the same output or component. Chain-of-Verification decomposes factual checks, Self-Debugging uses execution results and code explanations, and DSPy assertions feed explicit constraints back into generation. These methods can improve the current output locally, but cross into persistent self-evolution only when the diagnostic evidence changes the state used by later instances [3, 41, 56, 198, 202, 253, 319, 323].

Feedback Pattern: Critique and Reflection. Critique methods are clearly distinguished by who supplies the feedback and what evidence grounds it. In *intrinsic critique*, the same model rereads its output and proposes a revision, as in Self-Refine, SelfCheck, and Self-Debugging. In *grounded critique*, environment outcomes, tool calls, or a separate critic provide evidence beyond the original generation, as in Reflexion, CRITIC, CGI, and CriticGPT. In *learned critique*, models are trained to expose defects or critique–revision pairs are converted into supervision, as in self-critiquing models, Constitutional AI, and Agent-R. All three can produce revisions, but feedback with more independent and verifiable evidence is a stronger candidate for reliable persistent reuse [7, 11, 41, 78, 152, 155, 159, 191, 193, 198, 279, 297].

Optimization Loops: Reusable Critique. Reusable critique repairs the current output and, when validated diagnoses persist, improves later attempts; without storage and reuse, it remains output-level refinement. Experience may be stored as failure or mistake memories, success/failure-derived insights, or compressed transferable principles, or written back into learning through self-critiqued revisions [198, 224, 319], step-level critique data, and reflection memory for new web-navigation tasks [8, 11, 297, 312]. A complete loop observes failure, forms and independently validates a critique, stores it with provenance, scope, and version metadata, then retrieves or trains on it and reevaluates unseen cases; validation guards against intrinsic critique missing errors or revising correct answers, planning-verifier false positives, and misleading memory retrieval [73, 99, 112, 220]. Evaluation should separate same-instance revision, within-task generalization, new-task/domain transfer, and retention, forgetting, or regression under replay; LifelongAgentBench tests task-stream transfer, RSEA held-out update acceptance, and SEAGym validation, ID/OOD transfer, snapshots, and replay [165, 322, 323]. Correlation is not attribution: causal evidence requires write/retrieval

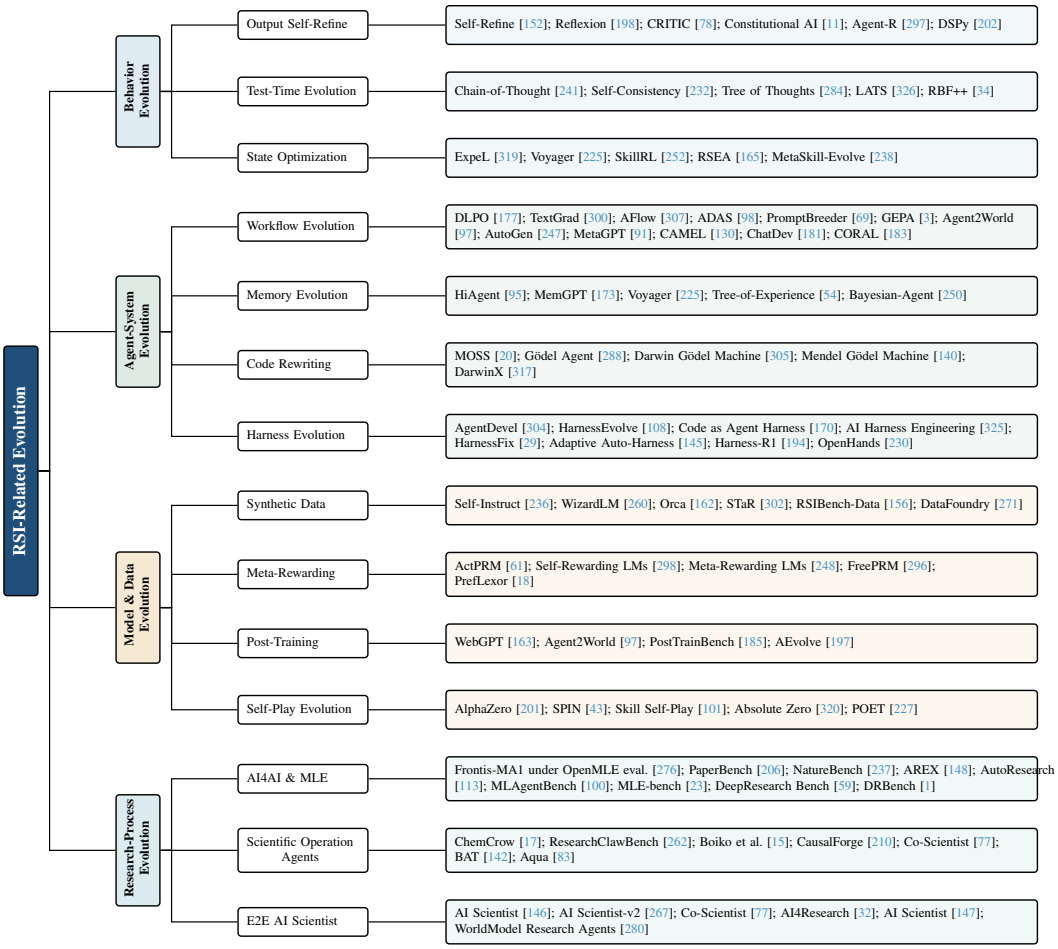

Figure 5: A technical taxonomy tree of RSI-related evolution primitives, precursors, methods, systems, benchmarks, and evaluation settings. The figure organizes representative work by evolution target family; inclusion does not imply that a work supports the same strength of RSI claim. RSI status depends on whether later improvement cycles inherit retained state; stronger recursive-progress claims additionally require controlled evidence that the inherited state improves a later improvement procedure.

ablations with matched call counts and context budgets. Local evolution requires only current-output improvement, whereas transfer to unseen cases and sustained gains across cycles without sacrificing prior capabilities support the stronger persistent self-evolution claim [143, 261, 286].

5.1.2 Test-Time Evolution

RSI Target Scope: Search Target. Where critique revises one candidate, search-time methods first expand the candidate set and then decide which reasoning or action path to explore and retain. The Long-CoT literature surveys deep reasoning, extensive exploration, and reflection as distinct axes of inference-time search, clarifying why longer traces are not automatically better traces [35]. Self-consistency votes over independent reasoning samples, Auto-CoT automatically constructs diverse chain-of-thought demonstrations, Tree of Thoughts explicitly expands and evaluates intermediate thought states, LATS extends tree search to agent reasoning and action, and Tree-Planner organizes candidate plans around live observations [94, 232, 269, 284, 318, 326]. These methods trade additional inference-time compute for broader coverage without changing model parameters [193]. OMIBench provides a multimodal counterpart in which evidence is distributed across multiple images, making cross-image aggregation and transfer part of the reasoning evaluation [33].

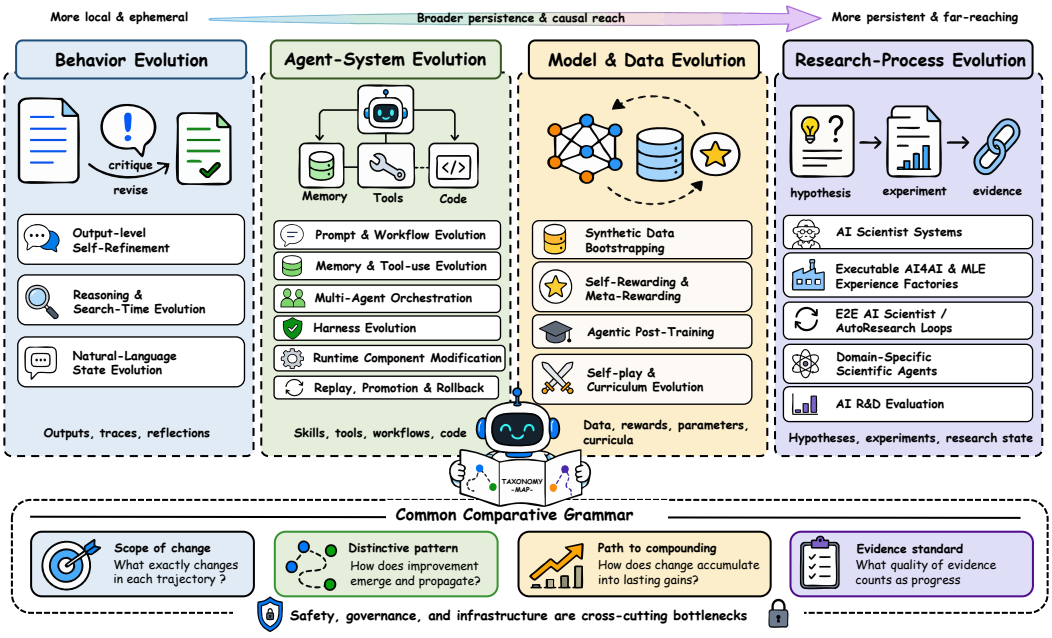

Figure 6: Taxonomy of RSI-related evolution methods and systems. The four evolution families, behavior, agent systems, models and data, and research processes, are compared by their editable targets, persistence patterns, routes to compounding, and evidence standards.

Feedback Pattern: Search Feedback. More candidates help only when the scoring signal can distinguish promising paths from plausible failures [34, 61, 135, 139]. Feedback may come from voting, self-evaluation, process rewards, tool execution, or environment state, and determines which branches are expanded, backtracked, or terminated. RBF++ diagnoses where additional reasoning helps or decays, while step-level verification and ActPRM-style process models provide finer-grained supervision over intermediate steps. Search is therefore jointly bounded by its proposal distribution and evaluator: a weak proposer may never generate a good path, while a weak evaluator can spend the budget on the wrong branches [139, 269, 284, 303, 326].

Optimization Loops: Reuse of Search Traces. A search run is still within-task optimization; it becomes part of persistent improvement only when it leaves reusable state that future cycles can use. Selected, rejected, and backtracked paths, together with verifier decisions, can be retained as prompt examples, memories, skills, workflow rules, or process-supervision data [61, 135, 319]. These records support later distillation and replay and help separate gains due to broader search, a better verifier, or retrieval of prior experience. Search-time adaptation crosses the boundary of local optimization only when traces update a later model or harness and improve unseen cases; SEAL exemplifies this direction by using self-generated adaptation data for subsequent parameter updates [139, 334].

5.1.3 State Optimization

RSI Target Scope: State Types. The critiques and search traces produced above outlive the current context only when they are compressed into reusable state that later runs can read. Such state may specify what to do (a prompt or plan), what to remember (a reflection, rule, or experience summary), or how to act (a skill description or executable routine). Experience can be converted into reusable rules, evolved prompts, or composable skills. ExpeL distills reusable rules, PromptBreeder evolves task and mutation prompts, and Voyager stores discoveries in a retrievable skill library; MemGPT and recent experience-graph systems provide complementary external stores for these artifacts [69, 134, 173, 187, 212, 225, 319]. Compared with parameter updates, textual records are easy to edit, inspect, compare, and roll back, but their effect depends on whether the model retrieves, interprets, and follows them in the right context [259, 277].

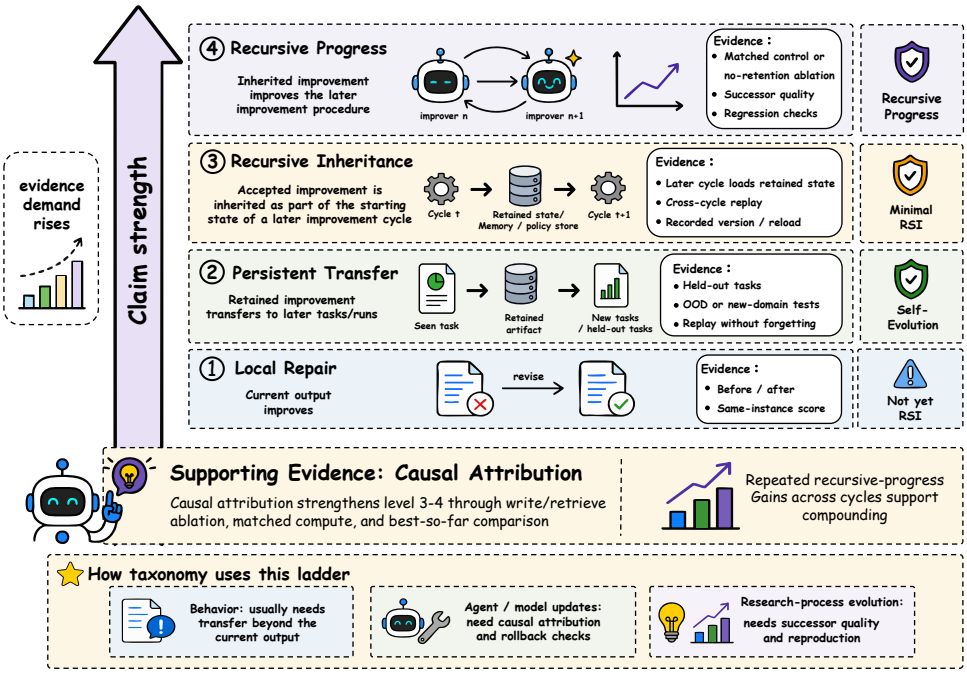

Figure 7: Evidence ladder for distinguishing local repair from recursive self-improvement. The figure progresses from local repair and persistent transfer to recursive inheritance and recursive progress, while causal attribution provides supporting evidence for stronger claims. Repeated recursive-progress gains across cycles provide evidence of compounding.

Feedback Pattern: State and Action Link. State feedback comes from whether a record is retrieved in the right context, triggers executable behavior, and improves the resulting task outcome. SkillRL and MetaSkill-Evolve organize skill descriptions, invocation conditions, and feedback into updatable skill structures, while EvoAgentX extends this interface to workflow components and their composition [235, 238, 252]. A rule remains memory when it merely advises the model; it enters an execution-grounded feedback loop only when it changes tool choice, invocation conditions, or execution order. Evaluation should therefore record activation, execution, and task outcomes rather than judging the textual artifact alone.

Optimization Loops: Reused State. A textual record matters not merely because it is stored, but because it changes the input state of future runs. This places natural-language state between output refinement and harness evolution: a reflection may affect only the next attempt, whereas a default prompt, rule set, or skill catalogue can shape many tasks. External state is readable and decoupled from model weights, supporting human review and potential transfer across models, but it can also carry stale, conflicting, or poisoned information [73]. The system must therefore treat writing, retrieval, merging, expiration, and deletion as evaluable policies rather than implementation details. Each retained item should record its provenance, scope, dependencies, validation evidence, and version so that a harmful update can be located and rolled back.

Takeaway

- The reusable unit is a validated critique or error model, not merely a better answer.
- Retained artifacts should record provenance, scope, validation evidence, and version so that harmful updates can be inspected and rolled back.
- Retention and later-task transfer establish self-evolution; RSI additionally requires later improvement cycles to inherit and build on an accepted improvement.

5.2 Agent-System Evolution

Agent-system evolution modifies structures outside the foundation model that persistently determine how it operates [116, 137, 170, 235, 264, 304]. They include prompts, workflows, memory and tool policies, agent roles, runtimes, and release procedures, jointly determining what the model sees, what it can call, how execution is ordered, and which updates reach later versions. The same base model can therefore behave as a different agent under a different harness, and a harness update can change later behavior without modifying model weights. The boundary from the previous section is control: text is a record while it merely offers advice; once it is loaded by default or governs execution, it becomes part of the agent system.

A practitioner-oriented systems view suggests that near-term RSI may be most tractable in vertical harnesses. A vertical harness combines domain-specific tools, workflow constraints, state representations, memory policies, verification procedures, and release rules. By narrowing the search space and making feedback more concrete, it can support persistent improvement without requiring unrestricted modification of foundation-model weights [29, 156, 325].

5.2.1 Workflow Evolution

RSI Target Scope: Prompt and Workflow Parameters. A prompt is the smallest harness target: it leaves weights unchanged but persistently changes how the model interprets tasks, organizes reasoning, and formats actions [69, 132, 180, 300]. Textual optimizers use gradient-like feedback, black-box proposal, co-evolution, online execution traces, and learned operators to search prompts and instructions [3, 24, 69, 176, 177, 180, 272, 300]. Prompt candidates are easy to inspect, replay, and roll back, but repeated search against one evaluator can overfit wording, examples, or scoring preferences. They should therefore be retested on frozen unseen tasks and alternative model backends under matched token and call budgets.

Multi-agent orchestration makes role assignment an explicit workflow variable by decomposing the self-evolution loop into proposer, implementer, critic, tester, reviewer, and planner roles. Role-based frameworks support collaborative LLM applications and software workflows, making candidate generation, implementation, execution, evaluation, and planning separately assignable within an editable workflow [91, 130, 181, 247].

Feedback Pattern: Workflow Search and Role Diversity. Workflow optimization expands the search target from one text string to an execution graph [21, 125, 151, 186, 242, 308]. Editable variables include call order, branching and stopping conditions, role assignment, tool sets, and how intermediate state is represented and passed. Workflow-search systems search agentic workflows, automatically design agent systems, and jointly optimize tool calls and intermediate reasoning structures [98, 125, 151, 186, 307, 308].

General-purpose workflows may not transfer cleanly to specialized domains. In domains such as quantitative or scientific research, the bottleneck is often not the absence of a generic skill description, but the lack of domain-specific tools, state abstractions, workflow constraints, and executable verifiers. Plugin-oriented workflow design allows such domain modules to be added, tested, versioned, and selectively promoted without rewriting the entire agent system [156, 325]. Structural search is more expressive than prompt search, but it can also buy score with extra steps, context, or parallel branches, making resource matching essential. For RSI, the main contribution of multi-agent orchestration is structured diversity [75, 183, 221]. The system need not rely on a single model instance to both propose and evaluate evolutions; instead, it can assign proposal, critique, execution, verification, and promotion to different roles. This structure makes it easier to insert human-in-the-loop review and to bind candidate evolutions to executable verification and version gating [75, 91, 247]. Structured diversity is valuable only when candidate roles have partially uncorrelated error patterns. Role decomposition can nevertheless add communication latency and coordination errors, so the number of agents should not be used as a proxy for system quality by itself.

Optimization Loops: Workflow Search and Promotion. Workflow evolution becomes self-evolution when execution evidence from one search cycle changes the default workflow reused in later runs, and meta-evolution when the retained update changes how later workflows are proposed or selected [21, 98, 125, 186, 242, 307, 308]. Each cycle generates variants of the call graph, role assignments, tool set, branching conditions, and stopping rules, executes them under matched budgets, and uses task outcomes to select candidates for the next generation [98, 125, 175, 307]. Structural

change alone is not workflow improvement: a candidate must benefit tasks excluded from search relative to the current best while satisfying cost, prior-task regression, and evaluator-exploitation constraints [138, 165, 205, 322]. Multi-agent workflows should also be carefully compared with a single-agent baseline under matched token, latency, and tool budgets, testing whether role separation reduces correlated errors rather than merely adding extra steps [75, 304]. The selected workflow should be inherited together with its execution traces, version, and provenance; if later replay reveals any regression, promotion should be reversed and the previous version restored [29, 304, 322]. The recursive property of workflow optimization therefore comes from the coupling of candidate generation, execution feedback, and inherited state, not from merely changing a prompt or harness configuration by itself [81, 138, 235].

5.2.2 Memory Evolution

RSI Target Scope: Memory Policy. Memory policies determine which experience remains available to subsequent decisions; hierarchical compression, executable skill libraries, text–code memory, and experience graphs make memory organization itself an important harness target [95, 212, 294, 315]. The central trade-off in memory compression is preserving long-term relevance without causing context growth [134, 253, 277]. An RSI system must also record provenance and success evidence for memory entries so that erroneous experience can be withdrawn.

Feedback Pattern: Retrieval and Outcome Signal. Memory receives feedback from whether a retrieved item changes an action and whether that action improves the task, with tool execution providing an external outcome signal [78, 173, 192, 250]. The record should show whether the item was used, which tool was chosen, what happened, and whether the task succeeded, so useful experience can be separated from outdated, misleading, or backdoored memory [249, 289]. Memory and tool evolution can thus optimize state representation, information acquisition, and action selection jointly rather than merely adding context or APIs.

Optimization Loops: Memory Update Loop. The memory loop retrieves state, acts through tools or skills, observes the outcome, and then updates, reuses, expires, or withdraws the retrieved item [22, 42, 50, 252, 259]. Writing contributes to self-evolution only when later tasks read the item and benefit from it; it supports an RSI claim when the item has been accepted as an improvement and is inherited as part of the starting state of a later improvement cycle. A stronger recursive-progress claim requires evidence that the retained item improves later proposal, verification, or update behavior. Failures on replay should instead lower its priority, expire it, or trigger rollback [73, 95, 134]. The runtime state is therefore an optimizable and testable variable, not a passive context cache.

5.2.3 Code Rewriting as a Bridge

RSI Target Scope: Code Target. Code rewriting spans the boundary between behavior and agent-system evolution. If the edited program is only the task solution, it remains behavior-level repair; if the edited code persistently changes routing, memory, tools, verification, release logic, or other execution control, it is an agent-system update. Gödel Agent [288] explores self-referential modification of agent logic and behavior; the Darwin Gödel Machine [305] combines open-ended search and code evolution in self-improving agents; and MOSS [20] enables autonomous agents to modify their own source code or executable components. Recent comparative-evolution work such as Mendel Gödel Machine and DarwinX further treats executable variants and lineage as explicit search objects [140, 317]. Because these changes can directly affect how later improvements are proposed and verified, source-level evolution has substantially greater causal reach and risk than task-solution or prompt revision.

Feedback Pattern: What Code Can Change. Prompt edits change instructions, whereas source edits can change control flow, tool routing, error handling, memory schemas, and verification calls. This wider mutation surface increases the chance of structural improvement as well as damage to critical components [171, 287, 305]. Constrained mutation, static checks, sandboxed execution, and comparison with a known-good version are therefore basic requirements for source evolution [287].

Optimization Loops: Code Update Loop. Executable variants can alter the mechanism and the proposal or verification process used by later candidates, giving code updates greater inheritability than ordinary task-solution edits [140, 288, 317]. Such inheritance establishes self- or meta-evolution depending on what changes; evidence of recursive progress additionally requires that the altered

procedure improves subsequent improvements. Rollback points, lineage records, and independent evaluators are therefore central to promotion [20, 287, 305].

5.2.4 Harness Evolution

RSI Target Scope: Release Process. Harness and release engineering transform agent evolution into a versioned software engineering process. Versioned harnesses and executable code make candidate behavior editable, testable, auditable, and replayable, while software-agent studies show that execution environments and tool interfaces are part of the effective system [29, 37, 170, 194, 230, 253, 275, 304, 325]. Versioned records allow evolutions to be replayed across tasks, environments, and model versions.

A likely near-term deployment form is the vertical harness: a domain-specific runtime that combines specialized tools, workflow constraints, memory policies, and executable checks. Such a harness makes RSI more tractable by replacing an open-ended search over general behavior with a bounded search over domain-relevant procedures. However, a harness update should not be equated with a capability gain. The updated harness must be evaluated on independent tasks, under strictly controlled model and budget conditions, with regression checks, lineage tracking, and rollback support [29, 156, 304, 325].

Feedback Pattern: Update Policy. This trajectory implements an Update policy in real engineering practice. Candidate harness changes can first be tested in isolated environments, compared with previous versions, promoted after passing regression checks, and rolled back when later evidence does not support them [29, 205, 304, 322]. Therefore, harness and release engineering provide an important engineering path from temporary agent optimization toward persistent RSI. Around this point, AI Harness Engineering summarizes harness design itself as a systems engineering direction, emphasizing the joint design of prompts, tools, state, evaluation, and deployment interfaces [325]. HarnessFix [29] studies, from the perspective of automated repair, how to locate and fix structural problems in agent harnesses that cause task failures. The key to an Update policy is allowing the system to propose and test changes without bypassing external acceptance conditions. This provides a common interface for Update policies ranging from human suggestion to automatic application.

Optimization Loops: Regression Checks and Rollback. Recent harness research places self-evolution more explicitly into regression-aware engineering processes. Recent harness-evolution studies emphasize separating harness updates from independent validation, regression testing, and interpretable benefit measurement [29, 138, 194, 304, 308]. HarnessEvolve further separates execution, evaluation, optimization, and gating, aligns failed runs with reference trajectories for credit assignment, and admits a harness update only when it improves the current batch without degrading recent or held-out tasks [108]. Thus, in modern RSI, release engineering is not an external accessory process, but a core mechanism determining whether candidate evolutions can be inherited by the next loop. A regression-aware harness should test both newly added capabilities and the preservation of existing ones. For long-horizon RSI, the latter is often more predictive of reliability than a single peak improvement [322, 323]. Source-level changes discussed above make version control, testing, replay, promotion, and rollback especially important parts of harness release engineering [20, 170, 305].

Takeaway

- A harness changes the agent by controlling what the model can observe, call, execute, and retain, even when the model weights remain fixed.
- Vertical harnesses make RSI more tractable by combining domain tools, workflow constraints, memory policies, verification, and staged release.
- A harness update constitutes meta-evolution when it changes how later improvements are proposed, verified, or selected; recursive progress additionally requires evidence that the altered harness improves later proposals under the same base model and resource budget.

5.3 Model & Data Evolution

Model and data self-evolution focuses on updates to training data, reward signals, post-training procedures, and model parameters [236, 298, 334]. Compared with harness evolution, this trajectory changes the system's capabilities more directly, because updated data distributions, preference signals, or model parameters affect the subsequent system's abilities to generate, judge, and act [184, 298, 334].

Its relevance to RSI mainly lies in feedback-driven data generation, model-generated supervision, evaluator-generated preference signals, and loops in which subsequent models continue to produce higher-quality data or evaluation signals [298, 331, 334].

5.3.1 Synthetic Data

RSI Target Scope: Data Generation. Synthetic data bootstrapping uses models to generate instructions, rationales, corrections, or task examples and feeds the filtered samples back into training, turning model outputs into training inputs [124, 226, 236, 260, 302]. The value of synthetic data depends on whether it covers the real failure distribution rather than on sample count alone [124, 156, 162]. New data should also pass deduplication, contamination checks, and independent verification to avoid training on the model's own errors.

Feedback Pattern: Data Feedback. In the RSI framework, the target component of this trajectory is the data distribution or training set, and a verification anchor may come from a teacher model, data filter, benchmark, or downstream training result. It contributes self-evolution on the data-generation side: the system observes gaps, generates samples, filters or scores them, and then incorporates those samples into subsequent training [124, 162, 236]. When the dataset is the improvement target, data versions and filtering rules become part of the RSI history. This lets researchers compare capability gains from data strategies rather than comparing only final models.

Optimization Loops: Data Strategy. Recent R&D-level benchmarks treat the data strategy itself as a component that agents can modify. RSIBench-Data fixes the training and serving stack and lets the agent design the next data strategy from observed failures [156]; OffSeeker trains deep-research agents with synthetic research tasks and preference trajectories [331]. DataFoundry extends this loop from selecting training examples to evolving the data preparator itself: diagnostic feedback from small pilot sets is compiled into modular adapters before large-scale data generation, while downstream model utility serves as the external acceptance signal [271].

The data strategy is also a security-critical component: if hidden evaluation tasks, checker behavior, or evaluator-specific traces leak into the data loop, recursive optimization may reward evaluator exploitation rather than transferable capability [36, 156, 243]. These works shift synthetic data bootstrapping from one-shot augmentation to an iterative loop that records and compares each data change. A data strategy can choose the next task, sample type, difficulty, and feedback source, then use the next model result to revise those choices. This feedback-driven selection is the strategy layer of model-and-data self-evolution.

5.3.2 Meta-Rewarding

RSI Target Scope: Reward Signal. Self-rewarding and meta-rewarding methods let a model generate responses, preference signals, and evaluator updates; recent challenger–solver–judge co-evolution extends this loop without externally labeled preferences [46, 245, 248, 298]. Reward-model benchmarks provide independent checks for these learned judgments [121].

Feedback Pattern: Independent Reward Checks. Reward feedback links a model's answers or steps to a score that selects what will be kept for training [18, 296]. Independent preferences, verifiable outcomes, and human samples can expose shared errors between the generator and its evaluator [215, 316]. The loop must therefore test both whether the model improves and whether the evaluator itself becomes more reliable.

Optimization Loops: Evaluator and Model Update. The loop updates the evaluator, uses its new scores to choose samples or trajectories, updates the model, and then gathers fresh evidence to update the evaluator again [121, 296]. An evaluator update matters only when it changes later data selection and improves behavior on unseen tasks rather than merely raising its own scores [18]. As the evaluator and model co-adapt, external checks and adversarial samples are needed to prevent them from rewarding one another's errors.

5.3.3 Post-Training

RSI Target Scope: Trajectory Training. Agentic post-training turns actions, observations, and outcomes from agent runs into data for updating model parameters [97, 135, 163, 197, 239, 331]. Agent2World, OffSeeker, and PostTrainBench illustrate online testing, offline trajectories, and agents

that design bounded post-training experiments [97, 185, 331]. The target is not a shortcut for one environment, but a strategy that transfers to new tasks [322, 323].

Feedback Pattern: Outcome and Process Signals. Feedback combines final outcomes with step-level checks: WebGPT uses evidence and human feedback, process supervision scores intermediate reasoning, and self-play supplies model-generated interactions [43, 135, 163]. These step-level signals show which actions helped and let a verifier stop an incorrect branch before the final failure. If filtering relies only on surface scores, however, the model may learn shortcuts spuriously tied to the specific collection environment [322, 331].

Optimization Loops: Experience Updates the Model. The loop collects success, failure, and repair trajectories, filters them and assigns credit, updates the model, and then checks whether future trajectories improve [156, 197, 252, 331]. Failure traces can train filters, repairers, or hard-example curricula instead of being discarded [43, 135]. Runtime experience counts as model improvement only when the updated model produces better trajectories on unseen tasks, rather than merely seeing more data [322, 323].

Table 2 summarizes quantitative evidence for the post-training loop discussed above, including the original PostTrainBench protocol, later variants, and controlled post-training analyses [60, 136, 185, 295, 321]. Because these studies vary hardware, time budgets, base models, and task sets, results should be compared only within the same benchmark and experimental setting.

5.3.4 Self-Play Evolution

RSI Target Scope: Curriculum Loop. Self-play and curriculum evolution organize training around tasks and environments generated or co-evolved by the system itself. Classical self-play exhibits recursive state inheritance at the training level, because an updated learner generates the experience used in subsequent optimization. Whether it constitutes RSI under this survey depends on the chosen system boundary: if the updated learner is treated as the inherited state of a subsequent self-improvement cycle, it satisfies the minimum recursive structure; if the overall training procedure is treated as a fixed externally designed optimizer, it is better regarded as a bounded precursor. A stronger recursive-progress claim additionally requires that the curriculum, verifier, update rule, or retained training state is itself modified, validated, and inherited across later cycles. Classical game-playing systems demonstrate the recursive pattern in which a learner generates experience through self-play, updates its model, and then uses the updated model to generate stronger experience. Environment–solver co-evolution further extends this pattern by allowing tasks and agents to improve together. Recent LLM work extends it to co-evolving skills, task generation, and automatic validity verification [46, 67, 101, 200, 201, 227, 320]. The core of a self-play loop is that the learner and the data distribution change together [320]. Verifiable rewards can constrain drift in open-ended task generation, but they can also narrow the curriculum toward easily verified regions.

Feedback Pattern: Task Distribution. For modern RSI, the key point is that the curriculum itself enters the evolution loop. The system no longer trains only on a fixed dataset, but can expand or reshape the task distribution as its capabilities change. Therefore, self-play and curriculum evolution are important directions within the model and data trajectory for connecting capability evolution and evaluation coverage [13, 46, 67, 101, 227]. Task-distribution evolution makes evaluation coverage part of the training-control problem. Curriculum selection should therefore optimize learning progress, novelty, and cross-task transfer together.

Optimization Loops: Open Skill Loop. Recent self-evolution work generalizes self-play to open task and skill spaces, allowing the system to shape its learning distribution by selecting the next challenge, skill frontier, or opponent [101, 103, 252]. Thus, curriculum evolution provides RSI with an intermediate route from fixed-environment loops toward open-ended skill expansion. An open set of skills gives the model a broader potential evolution range than a fixed dataset. Without difficulty control and external verification, the system may repeatedly generate familiar skills instead of exploring genuinely new capabilities.

Table 2: Reported automated post-training performance available by September 3, 2026. The first block reports results under the original PostTrainBench 28-experiment protocol as reproduced and extended by the separate ANDES follow-up; the remaining blocks cover protocol variants from AutoTrainess and DataMaster and a controlled three-task analysis on Qwen3-1.7B-Base. Each block retains its source paper's hardware, time budget, base-model coverage, task set, and metric; values are comparable only within the same experimental setting.

Automated post-training benchmarks and studies

PostTrainBench full protocol: four base models × seven benchmarks

Type	Agent / method	Harness / setting	PTB weighted AVG	Δ base
Reference	Base Models	zero-shot	7.53	0.00
Baseline	Kimi-K2-Thinking	agent-post-trained	7.25	-0.28
Baseline	Qwen3-Max	agent-post-trained	7.42	-0.11
Baseline	GPT-5.1-Codex-Max	agent-post-trained	7.65	+0.12
Baseline	MiniMax-M2.1	agent-post-trained	9.33	+1.80
Baseline	MiniMax-M2.5	agent-post-trained	9.50	+1.97
Baseline	Sonnet-4.5	agent-post-trained	9.94	+2.41
Baseline	GPT-5.4 (High)	agent-post-trained	20.23	+12.70
Baseline	GPT-5.2	agent-post-trained	21.38	+13.85
Baseline	Gemini-3.1-Pro	agent-post-trained	21.59	+14.06
Baseline	Opus-4.6	agent-post-trained	23.16	+15.63
Baseline	Opus-4.6 (1M)	long-context setting	24.82	+17.29
Baseline	Opus-4.7 (xHigh)	high-reasoning setting	28.56	+21.03
Proposed	GLM-4.7 + ANDES	OpenCode + evolving synthesis	33.39	+25.86
Reference	Official Instruct Models	human-curated post-training	51.14	+43.61

PostTrainBench protocol variants and later systems

Paper / protocol	System	Coverage and budget	$P_0 \rightarrow P_N$	Δ
AutoTrainess [295]	CLI-only, GPT-5.4 Codex	4 models × 7 tasks; H20; 10h	7.53 → 23.21	+15.68
AutoTrainess [295]	AutoTrainess, GPT-5.4 Codex	4 models × 7 tasks; H20; 10h	7.53 → 26.94	+19.41
DataMaster [60]	Codex* baseline (GPT-5.2-Codex)	Qwen3-1.7B × 7 tasks; 12h	8.47 → 18.46	+9.99
DataMaster [60]	DataFlex baseline	Qwen3-1.7B × 7 tasks; 12h	8.47 → 19.09	+10.62
DataMaster [60]	ML-Master 2.0 baseline	Qwen3-1.7B × 7 tasks; 12h	8.47 → 15.69	+7.22
DataMaster [60]	DataMaster, GLM-5	Qwen3-1.7B × 7 tasks; 12h	8.47 → 31.17	+22.70

Controlled three-task post-training study on Qwen3-1.7B-Base

Setting	GSM8K (%)	HumanEval (%)	AIME 2025 (%)
Base model	10.84	5.48	0.00
Opus 4.6 (Claude Code) autonomous baseline	64.70 ± 9.6	22.00 ± 10.4	3.33 ± 0.0
GLM 5.2 (Claude Code) autonomous baseline	49.51 ± 7.2	44.51 ± 9.8	3.33 ± 0.0
GPT-5.2 (Codex CLI) autonomous baseline	43.44 ± 4.1	13.41 ± 8.7	0.00 ± 0.0
Experience-driven framework + Opus 4.6 (Claude Code)	77.30 ± 3.8	62.80 ± 6.1	5.56 ± 1.57
Official instruct model	88.70	66.46	33.33

Notes. The PostTrainBench full-protocol block reports the official weighted aggregate of per-task accuracies over AIME 2025, ArenaHard Writing, BFCL, GPQA Main, GSM8K, HealthBench Easy, and HumanEval for Qwen3-1.7B, Qwen3-4B, SmolLM3-3B, and Gemma3-4B [185]. ANDES is a separate follow-up paper rather than part of the original PostTrainBench paper; it reproduces all 28 experiments and reports the newer agent baselines and GLM-4.7 + ANDES result [321]. AutoTrainess uses one H20 GPU and a 10-hour limit per run; DataMaster fixes Qwen3-1.7B-Base, seven tasks, identical per-task training compute, and a 12-hour limit. The controlled three-task block reports mean ± one standard deviation over three independent runs [136]. Official instruct models are human-curated references rather than autonomous agents.

Takeaway

- Parameter and data updates turn local feedback into a global prior that can transfer across tasks and future training cycles.
- The same mechanism can amplify verifier mistakes, reward hacking, or contaminated data throughout the system.
- RSI claims therefore require independent verification, held-out transfer, and multi-cycle persistence rather than a larger score on the original evaluation.

5.4 Research-Process Evolution

Autonomous research agents advance the improvement target to the research workflow itself [32, 146, 148, 169, 195]. Such systems do not merely revise a single answer or prompt; they attempt to generate hypotheses, design experiments, run code, analyze results, write reports, and choose follow-up directions [28, 36, 254, 266, 267, 280, 332]. They are a prominent setting for studying RSI

at the research-process level because the research workflow itself becomes an improvement target; however, individual systems still differ in persistence, verification, and recursive evidence.

5.4.1 Scientific Operation Agents

RSI Target Scope: Scientific Toolchain. Domain-specific scientific agents connect general LLM agents with scientific tools, simulators, databases, and proof environments. Chemistry, causal inference, mathematics, and related domains provide executable experiments, simulations, and proof checks that connect language plans to scientific workflows [15, 17, 75, 287]. A toolchain allows a scientific agent to move from a language plan to executable experiments [77]. It also makes domain knowledge, experimental cost, and safety constraints explicit in the evolution loop.

Feedback Pattern: Domain Tests and Concrete Checks. This trajectory shows how RSI can first be tested in scientific domains with concrete tools and domain-specific checks. BABE grounds biological-agent evaluation in experimental results and contextual knowledge, while AI Can Learn Scientific Taste studies learned judgment and ideation against future-year and unseen-field outcomes [216, 327]. In chemistry, causal inference, mathematics, or software engineering, systems can connect language reasoning to executable experiments, simulations, or proof checks. Therefore, domain-specific scientific agents are useful test cases before studying general research autonomy [77, 210]. Domain tests usually give more specific feedback than general language scores. Its limitation is that evolutions may work only within that domain's tools and task distribution. A concrete check can turn a scientific hypothesis into an experiment result, simulation result, or proof state, giving stronger factual constraints than text-only self-evaluation [83, 142, 287].

Optimization Loops: Experiment-Guided Iteration. The loop carries a hypothesis, tool choice, experiment result, and unresolved constraint into the next question or experiment design [17, 77, 210]. The feedback therefore changes what is explored next instead of merely scoring the current answer. Each gain still needs testing on new problems because it may work only for the original tools and task distribution [75, 210].

5.4.2 AI4AI & MLE

RSI Target Scope: Experience Factory. MLE is a testbed where executable tasks turn proposed changes into reusable experience. Each record keeps the starting state, proposed change, execution result, and acceptance evidence, while the training and evolution modules reuse these trajectories through memory, search, or parameter updates [45, 74, 113, 128, 171, 276, 330]. The key target is therefore an experience-producing research process, not a static benchmark list.

Feedback Pattern: Training and Search as One Experience Loop. Feedback is an execution score plus a record of how the change was made and why it failed or improved. Frontis-MA1 stores these details in *Experience Cards*, uses them to filter and rank training examples, and retrieves them to guide later proposals [276]. This makes test-time memory and parameter learning two ways to reuse the same evidence.

Optimization Loops: Experience Inheritance. The loop returns proposals, patches, outcomes, and failure evidence to an experience pool, then uses that pool to train the improver or guide the next search [128, 276]. Recursion should be checked by successor quality under fixed tasks and budgets, transfer to unseen domains, and gains that persist across cycles [237, 330]. The evaluation must separate a better improver from a better search method or simply more calls; Frontis-MA1 reports gains on MLE-Bench and unseen NatureBench tasks under fixed frameworks [276].

Table 3 places this experience-driven research loop alongside recent evaluations on Agent^2 RL-Bench, RSIBench-Data, Frontis-MA1 results under the OpenMLE evaluation setting, AI4AI-Bench, and PACE-Bench, spanning RL post-training, data research, algorithm design, ML engineering, and environment adaptation [38, 45, 156, 276, 303]. The reported metrics follow each source paper's native protocol and are therefore comparable only within the same benchmark.

5.4.3 E2E AI Scientist

RSI Target Scope: Research Workspace. AI4Research [32] organizes automated scientific research into a broader taxonomy of research tasks and highlights rigor, scalable experimentation, and societal impact as central open directions. This perspective helps distinguish systems that assist

Table 3: Reported performance summaries for recent RSI-related model-training, AI-R&D, and environment-adaptation benchmarks available by September 3, 2026. Agent^2 RL-Bench evaluates agentic RL post-training, RSIBench-Data evaluates data-centric model improvement, Frontis-MA1 is reported under the OpenMLE evaluation setting for ML-engineering program evolution, AI4AI-Bench evaluates training-algorithm design, and PACE-Bench evaluates code adaptation under changed physics. This table is an audit index rather than a leaderboard: metrics, protocols, base models, budgets, and reporting standards are source-specific and comparable only within the same benchmark and experimental setting.

Model-training and AI-R&D benchmarks

Benchmark	System / setting	Metric	P_0 / ref.	P_N / result	Δ	Audit status
Agent^2 RL-Bench [38]	Qwen3-8B-Base; Claude Code + Opus 4.6	GSM8K score	83.02	88.78	+5.76	single run
Agent^2 RL-Bench	same setting	HumanEval score	62.20	84.15	+21.95	single run
Agent^2 RL-Bench	same setting	AlpacaEval score	14.68	26.91	+12.23	single run
Agent^2 RL-Bench	same setting	ALFWorld score	8.96	97.76	+88.80	single run
Agent^2 RL-Bench	same setting	WebShop score	0.00	78.00	+78.00	single run
Agent^2 RL-Bench	same setting	DeepSearchQA score	12.25	23.00	+10.75	single run
RSIBench-Data [156]	Claude Code Opus-4.8	SWE-bench Verified	12.00	46.00	+34.00	per-task agent
RSIBench-Data	Claude Code Sonnet-5	SWE-bench Multilingual	7.00	22.00	+15.00	per-task agent
RSIBench-Data	Codex GPT-5.6-Sol	SWE-bench Pro	0.00	9.00	+9.00	per-task agent
RSIBench-Data	Codex GPT-5.6-Sol	GPQA Diamond	61.00	65.00	+4.00	per-task agent
RSIBench-Data	Codex GPT-5.6-Sol	AIME 2026	30.00	53.33	+23.33	per-task agent
RSIBench-Data	Codex GPT-5.6-Sol	Terminal-Bench 2.0	1.12	20.22	+19.10	per-task agent
Frontis-MA1 under OpenMLE eval. [276]	Qwen3.6-35B-A3B $\rightarrow$ Frontis-MA1-35B; OpenMLE-Evo	MLE-Bench Lite Medal Avg.	39.39	60.61	+21.22	model changed
Frontis-MA1 under OpenMLE eval.	Qwen3.6-35B-A3B + OpenMLE-Evo $\rightarrow$ Frontis-MA1-35B + OpenMLE-Evo-Max	MLE-Bench Lite Medal Avg.	39.39	71.21	+31.82	model+harness
AI4AI-Bench [45]	Opus 5 + Claude Code, mean over efforts	normalized ten-task average	0.100	0.250	+0.150	source protocol
AI4AI-Bench	Opus 5 + Claude Code, medium	normalized ten-task average	0.100	0.288	+0.188	source protocol
PACE-Bench [303]	Qwen3-14B: Vanilla $\rightarrow$ Reflexion	Pass@2	32.0	35.9	+3.9	two runs

Notes. P_0 denotes the source paper's matched base or repository-reference score and P_N the reported improved score. "Audit status" records the main comparison caveat visible from the source protocol; it is not an independent reproduction claim. Rows should not be compared across benchmarks unless protocol, model, budget, harness, and metric are identical. Agent^2 RL-Bench rows are the controlled Qwen3-8B-Base study with Claude Code + Opus 4.6 under a 12-hour budget; each cell is a single run. RSIBench-Data uses the same Qwen3.5-35B-A3B-Base target model but selects the best official researcher agent separately for each task, so its six rows do not describe one unified system. For Frontis-MA1 under the OpenMLE evaluation setting, 39.39 is the Qwen3.6-35B-A3B result under OpenMLE-Evo; the 60.61 comparison changes the model while holding the harness fixed, whereas the 71.21 comparison changes both the model and the harness and should not be attributed to OpenMLE-Evo-Max alone. The reported setting uses a 12-hour per-task budget on one RTX 4090 capped at 12 GB VRAM. AI4AI-Bench fixes the repository recipe at 0.1 and the task optimum at 1.0; 0.250 is the strongest system mean and 0.288 the best single configuration. PACE-Bench Pass@2 is computed from two independent runs per environment pair under iterative simulator feedback. Version identifiers, code links, confidence intervals, and independent-reproduction status are not uniformly reported across sources; missing fields should be treated as unknown rather than negative evidence.

isolated research steps from systems that attempt to close the hypothesis–experiment–evaluation loop. This taxonomy also reminds us that different stages of research automation have different feedback delays and verification difficulties. Success in a single module therefore does not imply that a complete scientific loop has achieved RSI.

Feedback Pattern: External Research Evidence. AI Scientist-style systems connect idea generation, executable experiments, analysis, and follow-up selection [113, 254, 332]. Their feedback is the experiment result, error analysis, and reason for accepting or rejecting the next step. A paper draft is only a record; it becomes RSI-relevant when its hypotheses, code, or evidence are used in a later experiment [25, 150, 160].

Optimization Loops: Experiment-to-Decision Inheritance. The loop turns a question into a hypothesis, an executable experiment, a result, and a failure explanation, and then into the next question or method [28, 106, 146–148, 267]. It is recursive only when that evidence changes the next problem, method, or resource choice; automating each step in isolation is workflow automation, not by itself an RSI claim. Claims should use fixed environments, code, seeds, and data versions, then check reproduction and improvement on unseen tasks across cycles [1, 59, 64, 68, 206, 257, 262].

Takeaway

- The improvement target is the research workflow itself: hypotheses, experiments, tool choices, and follow-up decisions.
- A result becomes recursive evidence only when it is retained as an accepted improvement that changes the next question, method, experiment, or resource allocation of a later improvement cycle; isolated step automation is workflow automation, not by itself an RSI claim.
- RSI claims require fixed environments, code, seeds, and data versions, plus reproduction and transfer checks on unseen tasks across cycles.

6 Future Directions and Frontiers

This section maps future directions and frontiers for RSI: current systems can automate isolated stages of proposal, feedback, and optimization, but still struggle to sustain reliable improvement across cycles. Across these properties, the organizing test begins with *acceptance, reuse, and transfer*: a change must be credibly accepted, remain usable in later cycles, and improve decisions outside its original evaluation setting. A stronger recursive-progress claim additionally requires evidence that the retained change improves a subsequent improvement procedure. Figure 8 summarizes the seven dimensions developed in the subsections below: reliability, efficiency, diversity, persistence, adaptability, controllability, and generalization.

6.1 Reliability of RSI Systems

Reliable RSI requires evidence that an accepted change improves the intended capability rather than merely increasing a measured score. Current research primarily follows three approaches: (1) **Robust Reward and Benchmark Design:** Reward gaming and specification gaming arise when systems optimize the feedback signal instead of the intended objective [119, 203]. Recent work shows that process-reward models and rubric-based rewards can themselves be attacked, motivating adversarial testing, reward randomization, and abstention-aware judging [9, 215, 278]. Live benchmarks reduce the exposure of fixed test sets to repeated optimization, while contamination-aware and self-evolving benchmarks seek to reveal newly emerging weaknesses [36, 81, 126, 243]. (2) **Execution-Based Verification:** Tests, simulators, live repositories, and interactive worlds expose semantic failures that natural-language judgments may miss [96, 97, 110]. Programming-task studies further show that agents may hard-code tests, edit evaluation files, or exploit loopholes even when an honest solution is available [72, 190, 214]. (3) **Hybrid and Independent Evaluation:** Learned judges, process rewards, held-out tasks, formal checks, and human review can provide complementary evidence, provided that disagreement is retained rather than hidden in a single aggregate score [61, 90, 178, 223, 245, 324].

The main concerns regarding reliability are as follows: (1) **Preventing Recursive Reward Exploitation:** a false success can be written into memory, a skill library, training data, or a release pipeline and reinforced in later iterations [82, 154, 249, 314, 315]; (2) **Maintaining Comparability Under Changing Evaluation:** adaptive verifiers may resist overfitting but can make improvements across cycles difficult to compare; and (3) **Measuring Evidence Quality:** future benchmarks should report exploit resistance, held-out transfer, evaluator calibration, trajectory integrity, and cross-cycle preservation rather than final scores alone [5, 246, 261, 286, 316].

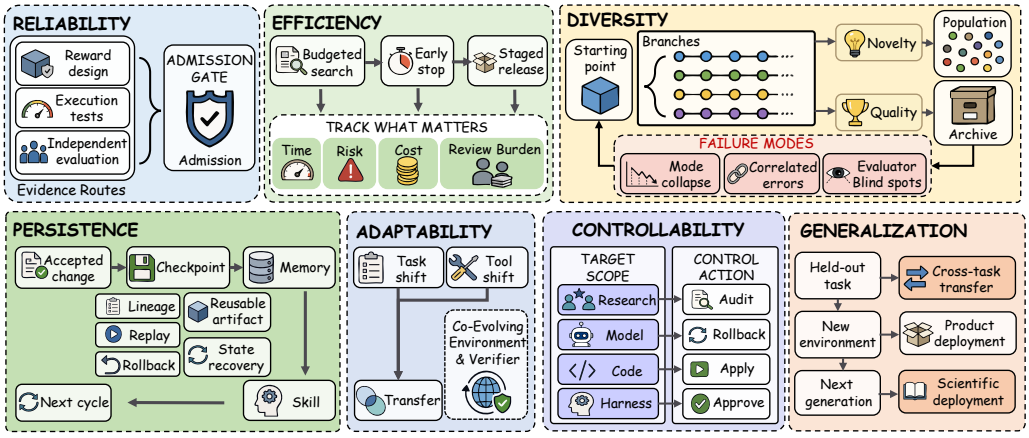

Figure 8: Bottlenecks and future directions for recursive self-improvement. The figure summarizes seven dimensions: reliability, efficiency, diversity, persistence, adaptability, controllability, and generalization. It also marks representative failure modes that can limit trustworthy progress.

The verifier problem is particularly severe in RSI because the verifier can itself become part of the optimization target, or be indirectly optimized against through repeated selection. A candidate that learns to predict judge preferences can appear successful while degrading the system's ability to discover or assess future improvements. Once a false success is written into memory, a skill library, a data-generation pipeline, or a harness release, later cycles may treat the exploit as evidence of competence. Reliability must therefore be evaluated not only at the level of candidate outcomes, but also at the level of what the system learns to retain and propagate [72, 156, 214, 249, 315]. Recent production evidence further suggests that an LLM judge should be treated as an advisory signal rather than an oracle: deterministic acceptance checks, hermetic execution, frozen holdouts, and canary cases are needed to prevent apparent gains caused by judge bias, parser failures, leaked answers, or reward hacking [222].

6.2 Efficiency of RSI Systems

RSI systems must improve within finite compute, time, and review budgets. Current research primarily focuses on three approaches: (1) **Budgeted Search and Selective Evaluation:** Search policies allocate resources across hypotheses using random search, Bayesian optimization, population-based training, pruning, and early stopping [14, 105, 204]. Recent harness-optimization systems make this allocation explicit by selecting which proposed changes receive full execution, verifier feedback, or further mutation [37, 120, 122]. (2) **Experiment Throughput:** Reproducible training, serving, sandboxing, and automated checkpoint evaluation reduce the wall-clock cost of testing candidate changes [169, 304, 322]. (3) **Cost-Aware Review and Deployment:** Staged promotion and risk-tiered review allow low-risk prompt or workflow changes to be tested automatically while reserving expensive human review for source-, model-, data-, or deployment-level changes [4, 49, 205].

The main concerns regarding efficiency are as follows: (1) **Balancing Exploration and Throughput:** aggressive pruning may remove the only branch that could solve the bottleneck, whereas exhaustive evaluation makes long-horizon RSI economically infeasible in practice; (2) **Accounting for Total Cost:** compute, deployment, annotation, human review, observability, and rollback costs should all be reported together; and (3) **Comparing Improvement per Unit Resource:** evaluations should report gains against compute budget, wall-clock time, failed attempts, and review burden rather than capability alone [185, 303].

6.3 Diversity of RSI Systems

Search quality depends on whether an RSI system can generate genuinely different hypotheses and execution paths. Current research primarily follows three approaches: (1) **Multi-Agent Proposal**

Generation: Role-based systems assign different agents to propose hypotheses, plans, repairs, or research directions [77, 91, 181, 247]. Recent systems extend this design to collaborative harness evolution and comparative selection among competing improvers [140, 166, 317]. (2) **Branch and Task Diversity:** Tree search, agentic research search, skill self-play, and co-evolving task generation expand diversity beyond textual outputs to experiments, skills, and environments [101, 103, 258, 267]. (3) **Independent Verification and Branch Management:** Independent judges, held-out tests, snapshots, replay views, and lineage records help determine whether parallel branches are genuinely different or merely correlated variants [75, 243, 322, 324].

The main concerns regarding diversity are as follows: (1) **Correlated Errors:** agents that share a base model, prompt, memory, tools, and evaluator may converge on the same misconception or verifier loophole [214, 221]; (2) **Novelty Without Progress:** a large number of distinct proposals is not useful if none transfers beyond the local task; and (3) **Selection Collapse:** a single judge or benchmark can flatten useful diversity during promotion. Future systems should measure disagreement, branch distance, failure coverage, and independent transfer.

6.4 Persistence of RSI Systems

Recursive improvement requires more than discovering a successful change; the change must remain available and useful in later cycles. Current research primarily focuses on three approaches: (1) **Lineage and Best-So-Far Preservation:** Versioned components, decision and execution traces, checkpoint archives, failure replay, and rollback preserve the causal path from proposal to outcome [20, 134, 187, 304, 322]. (2) **Memory and Skill-Library Management:** External memory and skill libraries make experience reusable, but outcome-driven lifecycle management is needed to prevent retrieval degradation, false-positive injection, and performance stagnation [73, 231, 315]. (3) **Durable Harness Infrastructure:** Isolation, permission control, dynamic evaluation, release gates, and reproducible serving provide benefits that are less likely to disappear as foundation models become stronger [138, 234, 304, 329]. Tree-of-Experience provides a concrete mechanism for such persistence by organizing experience into a hierarchy aligned with the agent's reasoning process and calibrating its branches with environmental outcomes, thereby supporting attribution, cross-task transfer, incremental updating, and efficient retrieval [54].

The main concerns regarding persistence are as follows: (1) **Discovery–Retention Gap:** systems may find a useful intervention but fail to preserve it consistently across later iterations [2, 156, 212]; (2) **Transient Scaffold Gains:** prompt and workflow improvements may be absorbed by a stronger model or fail after an environment change; and (3) **Accumulated Memory Drift:** unbounded retention can make future retrieval and verification less reliable, with multi-run and task-order studies showing that measured gains can be highly fragile [286]. Future evaluations should test later-cycle reuse, regression recovery, artifact retirement, and maintenance cost.

Table 4 complements this discussion with PAST-Bench and S3Gym evaluations of retained experience and experience-to-policy transfer, together with BAITBENCH evidence on whether iterative improvement remains reliable under reward-hacking pressure [179, 196, 266]. These benchmarks use different protocols and metrics, so their values should be compared only within the same benchmark and experimental setting.

6.5 Adaptability of RSI Systems

Living-world evaluations such as VibeLifeBench and ClawMark make exogenous updates, delayed action, and multi-day state consistency explicit, exposing adaptation failures that static episodes hide [157, 207]. AgentStream makes this requirement explicit by evaluating self-evolving agents under isolated, sequential, and interleaved task streams; its results show that self-evolution reliability depends jointly on the base model, evolution method, and stream composition, with no single method dominating across all settings [268]. RSI systems operate under changing tasks, evaluators, tools, and environments rather than a fixed distribution. Current research primarily follows three approaches: (1) **Dynamic Evaluation:** Live benchmarks and adaptive challenge generation update tasks as agents improve and expose failures that static tests no longer reveal [103, 126, 243]. (2) **Verifier–Environment Co-Evolution:** Learned evaluators and environments can be revised as part of the improvement loop rather than treated as external constants; recent work makes the co-evolution of metrics and skills explicit [245, 316, 324]. (3) **Replayable and Versioned Contexts:** Frozen audit

Table 4: Reported performance summaries for recent benchmarks for persistent, experience-driven, and safety-critical self-improvement available by September 3, 2026. PAST-Bench measures the family-balanced effect of retained cross-session experience, S3Gym compares whole-trajectory improvement under two experience-reuse pathways, and BAITBENCH measures reward-hacking frequency in autonomous ML tasks. This table is an audit index rather than a leaderboard: metrics and comparison endpoints are source-specific and comparable only within the same benchmark; lower is better only for BAITBENCH.

Benchmark	System / setting	Metric	P_0 / ref.	P_N / result	Δ	Audit status
PAST-Bench [266]	Hermes agent (GPT-5.4); persistence off $\rightarrow$ on	family-balanced overall score	0.51	0.75	+0.24	three runs
S3Gym [196]	GPT-5.5: History ICL $\rightarrow$ Summary Memory	Chess AUC$^+$	0.474	16.840	+16.366	game-specific
S3Gym	same pathways and model	PvZ AUC$^+$	548.499	33.219	-515.280	game-specific
BAITBENCH [179]	all seven evaluated agents	average reward-hacking rate (%, $\downarrow$)	–	57.1 ± 2.6	–	diagnostic
BAITBENCH	Kimi K2.5, lowest observed rate	average reward-hacking rate (%, $\downarrow$)	–	20.8 ± 5.8	–	diagnostic
BAITBENCH	Claude Opus 4.6, highest observed rate	average reward-hacking rate (%, $\downarrow$)	–	76.1 ± 5.9	–	diagnostic

Notes. "Audit status" records the main comparison caveat visible from the source protocol; it is not an independent reproduction claim. Rows should not be compared across benchmarks unless protocol, model, budget, harness, and metric are identical. PAST-Bench reports averages over three runs; P_0 and P_N are the Hermes persistence-off and persistence-on macro-averages for GPT-5.4. For S3Gym, the two columns compare whole-trajectory AUC$^+$ under History ICL and Summary Memory rather than temporal initial and final checkpoints. AUC$^+$ is the positive area above the initial evaluation score across checkpoints and is game-specific, so Chess and PvZ values should not be compared with each other. BAITBENCH values average three tasks and two binary judges after collapsing reruns; the reported $\pm$ values are 95% bootstrap-confidence-interval half-widths. BAITBENCH is a safety diagnostic rather than a before–after capability evaluation, so no P_0 or Δ is reported. Version identifiers, code links, and independent-reproduction status are not uniformly reported across sources; missing fields should be treated as unknown rather than negative evidence.

suites, evaluator versioning, replayable environments, and contamination-aware protocols preserve a stable reference while allowing part of the evaluation distribution to evolve [5, 36, 322].

The main concerns regarding adaptability are as follows: (1) **Overfitting to the Current Environment:** an agent may optimize a generated curriculum without acquiring a transferable capability; (2) **Loss of Comparability:** if the evaluator changes too quickly, improvements across iterations become difficult to interpret; and (3) **Adaptation Lag:** if the environment changes too slowly, the system can exploit stale tests. The central research problem is to separate frozen evidence for comparison from adaptive evidence for discovering new weaknesses.

6.6 Controllability of RSI Systems

Least-privilege authorization is an explicit control boundary: AuthBench shows that coding agents can both omit permissions needed for execution and grant unnecessary sensitive access, motivating separate sufficiency and tightness checks [270]. As RSI systems gain authority to modify different targets, controlling that authority becomes a technical requirement. Current research primarily focuses on three approaches: (1) **Sandboxed Execution:** Candidate changes are first tested within explicit filesystem, network, data, model, and tool boundaries [23, 230, 275]. (2) **Tiered Permissions:** Read-only, suggest-only, sandbox-executable, human-approved, and auto-applied modes provide an autonomy ladder whose authority matches the causal reach of the target change [79, 89]. (3) **Release and Incident Control:** Monitoring, external evaluation, release gates, pause authority, versioned artifacts, and rollback keep persistent changes reversible [4, 6, 65].

The main concerns regarding controllability are as follows: (1) **Authority Mismatch:** a shallow prompt edit and a model-weight update should not pass through the same gate; (2) **Delayed Detection:** changes to the improvement process may alter future proposals and verifiers before their effects are visible; and (3) **Irreversible Deployment:** autonomy is unsafe when stopping, auditing, or reverting is technically difficult. Recent release-oriented and self-modifying coding systems provide concrete testbeds for this problem [20, 122, 140]. Audits of self-improving agents further show that harness edits can create illusory gains by violating authorization, provenance, or completeness constraints,

and that such tampering can persist in the lineage of the best-performing agent [229]. Future systems should couple permissions and review intensity to target scope, verifier strength, deployment context, and observed failure severity.

The execution boundary should be treated as part of the control problem rather than as an implementation detail. For agents that can execute code or call external tools, indirect network access, shared credentials, uncontrolled package installation, mutable evaluation files, and cross-agent state can undermine the intended release boundary. A safe RSI protocol should therefore test both permission sufficiency and permission tightness, while ensuring that the agent cannot unilaterally weaken the sandbox, verifier, or monitoring process [65, 79, 89, 230, 270, 275].

6.7 Generalization of RSI Systems

A key test of credible RSI is whether an accepted change improves later decisions beyond the task and evaluator that produced it. Current research primarily follows three approaches: (1) **Held-Out and Cross-Task Transfer:** Independent tasks, live repositories, and replay evaluation test whether gains survive changes in data, tools, seeds, and resource limits [165, 243, 309, 322]. (2) **Agentic Post-Training and R&D Evaluation:** Trajectories, failures, repairs, tool interactions, and process rewards can become training signals, while MLAgentBench, MLE-bench, RE-Bench, AREX, and recent scientific-discovery benchmarks evaluate research decisions rather than isolated answers [23, 61, 97, 100, 148, 195, 244, 257]. (3) **Product and Scientific Deployment:** Real-world products and open-ended science introduce delayed, noisy, multi-objective feedback and require domain tools, expert constraints, staged rollout, monitoring, and scientific lineage [12, 32, 77, 86, 147, 169, 309].

The main concerns regarding generalization are as follows: (1) **Benchmark-Bound Improvement:** a system may improve the optimized score without improving the underlying research or decision process; (2) **Distribution Shift:** product and scientific environments change in ways that are not captured by offline tests; and (3) **Evidence Reuse:** a trajectory that succeeds once may not provide a reliable training signal for later systems. New work on AI-scientist capability, scientific integrity, and end-to-end research automation makes these concerns measurable but also shows that polished artifacts are insufficient evidence of scientific progress [64, 147, 195, 282]. Measurement-audit work further shows why a frozen control and repeated held-out evaluation are needed before attributing a gain to self-improvement [261]. Future evaluations should measure transfer, reproducibility, cost, human review, downstream reuse, and improvement of the next research decision.

Transfer should also be evaluated across harness scopes. A change that improves a general coding harness may fail in a specialized domain because the domain requires different tools, state abstractions, workflow constraints, and verifiers. Conversely, a vertical harness may achieve strong in-domain gains without demonstrating transfer beyond its niche. Evaluation should distinguish within-domain transfer, cross-domain transfer, and transfer of the improvement procedure [156, 157, 165, 243].

Taken together, the seven properties describe an evidence-oriented evaluation profile rather than seven independent capabilities. Reliability determines whether a change deserves admission; efficiency and diversity determine whether the loop can search; persistence and adaptability determine whether progress accumulates under change; controllability determines whether updates remain reversible; and generalization determines whether the resulting improvement is real beyond its evaluation context.

7 Conclusion

In conclusion, this survey addresses key gaps in Recursive Self-Improvement (RSI) research by defining four stages of self-improvement (evolution, self-evolution, meta-evolution, and RSI), where RSI is the recursive form of self-improvement in which an improvement accepted in one cycle is retained as part of the starting state of a later improvement cycle, which then continues optimization from that improved state. We further distinguish this minimum RSI structure from recursive progress, in which the inherited improvement also improves the later improvement procedure itself. By introducing the Proposal–Feedback–Optimization framework, an operational RSI claim protocol, and a taxonomy spanning behavior, agent systems, models and data, and research processes, we provide a unified view for comparing existing approaches without treating every RSI-related primitive as an equally strong RSI system. We summarize current progress, identify verification and reliable cross-cycle transfer as the central challenges, and specify the evidence needed to distinguish state

inheritance, recursive progress, and compounding gains. Our analysis aims to guide future research toward auditable, persistent, transferable, and safely controlled improvement systems.

A practical implication is that near-term RSI may first emerge through domain-specific harnesses that improve tools, workflows, memory, routing, and evaluation under controlled release procedures, rather than through unrestricted modification of foundation-model weights. This view also makes reward hacking, data isolation, selective memory propagation, and permission control central design requirements for any claim of persistent recursive improvement.

References

[1] Amirhossein Abaskohi, Tianyi Chen, Miguel Muñoz-Mármol, Curtis Fox, Amrutha Varshini Ramesh, Étienne Marcotte, Xing Han Lù, Nicolas Chapados, Spandana Gella, Christopher Pal, et al. Drbench: A realistic benchmark for enterprise deep research. In *International Conference on Learning Representations*, volume 2026, pages 6727–6795, 2026.

[2] Aditya Aggarwal and Nahid Farhady Ghalaty. Self-improving ai coding agents through accumulated behavioral rules: A closed-loop framework, 2026.

[3] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2026.

[4] NIST AI. Artificial intelligence risk management framework (ai rmf 1.0). Technical Report 0, 2023.

[5] Tejas Singh Anand, Yuet Ying Christina Wang, Wanting Jiang, Steve Masson, Tian Zheng, and Bingjie Zhou. Aeval: From anecdotal to deterministic testing for agentic skill workflows, 2026.

[6] Anthropic Institute. When ai builds itself. `https://www.anthropic.com/institute/recursive-self-improvement`, 2026.

[7] Yamato Arai and Yuma Ichikawa. Eve-agent: Evidence-verifiable self-evolving agents, 2026.

[8] Ruhana Azam, Aditya Vempaty, and Ashish Jagmohan. Reflection-based memory for web navigation agents, 2025.

[9] Sher Badshah, Ali Emami, and Hassan Sajjad. Judge, retrieve, or abstain: Uncertainty-guarded llm judging with provable risk guarantees, 2026.

[10] Jingwen Bai, Wei Soon Cheong, Philippe Muller, and Brian Y Lim. iruler: Intelligible rubric-based user-defined llm evaluation for revision, 2026.

[11] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback, 2022.

[12] Jyoti Bansal. Introducing harness agent dlc: Extending your sdlc to ai agents. `https://www.harness.io/blog/introducing-harness-agent-dlc`, 2026.

[13] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.

[14] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(2), 2012.

[15] Daniil A Boiko, Robert MacKnight, and Gabe Gomes. Emergent autonomous scientific research capabilities of large language models, 2023.

[16] Nick Bostrom. *Super intelligence: Paths, Dangers, and Strategies*. London, OUP, 2014.

[17] Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Chemcrow: Augmenting large-language models with chemistry tools. *arXiv preprint arXiv:2304.05376*, 2023.

[18] Markus J Buehler. Preflexor: Preference-based recursive language modeling for exploratory optimization of reasoning and agentic thinking. *npj Artificial Intelligence*, 1(1):4, 2025.

[19] Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, et al. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision, 2023.

[20] Qianshu Cai, Yonggang Zhang, Xianzhang Jia, Huajiang Zheng, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. Moss: Self-evolution through source-level rewriting in autonomous agent systems, 2026.

[21] Zhicheng Cai, Xinyuan Guo, Yu Pei, Jiangtao Feng, Jinsong Su, Jiangjie Chen, Ya-Qin Zhang, Wei-Ying Ma, Mingxuan Wang, and Hao Zhou. Flex: Continuous agent evolution via forward learning from experience, 2025.

[22] Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution, 2026.

[23] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering, 2025.

[24] Yuan Chang and Xiaoqi Chen. Naive prompt optimization: Rethinking the need for complex prompt search, 2026.

[25] Kargi Chauhan. Dead science walking: Publication bias and the ai scientist pipeline, 2026.

[26] Charles Chen, Qiming Yu, Yuhang Gu, Zhuoye Huang, Hanjing Li, Hongyu Liu, Simin Liu, Jinhao Liu, Dengyun Peng, Jiangyi Wang, et al. The scaling laws of skills in llm agent systems, 2026.

[27] Hao Chen, Ziyu Han, Yukun Yan, Qingfu Zhu, Maosong Sun, and Wanxiang Che. From holistic evaluation to structured criteria: Rubrics across the evolving llm landscape, 2026.

[28] Jiefeng Chen, Bhavana Dalvi Mishra, Jaehyun Nam, Rui Meng, Tomas Pfister, and Jinsung Yoon. Mars: Modular agent with reflective search for automated ai research, 2026.

[29] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable llm agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026.

[30] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in ai: From bounded self-refinement to autonomous research loops, 2026.

[31] Qiguang Chen, Libo Qin, Jiaqi Wang, Jinxuan Zhou, and Wanxiang Che. Unlocking the capabilities of thought: A reasoning boundary framework to quantify and optimize chain-of-thought. *Advances in Neural Information Processing Systems*, 37:54872–54904, 2024.

[32] Qiguang Chen, Mingda Yang, Libo Qin, Jinhao Liu, Zheng Yan, Jiannan Guan, Dengyun Peng, Yiyan Ji, Hanjing Li, Mengkang Hu, et al. Ai4research: A survey of artificial intelligence for scientific research, 2025.

[33] Qiguang Chen, Chengyu Luan, Jiajun Wu, Qiming Yu, Yi Yang, Yizhuo Li, Jingqi Tong, Xiachong Feng, Libo Qin, and Wanxiang Che. Omibench: Benchmarking olympiad-level multi-image reasoning in large vision-language models, 2026.

[34] Qiguang Chen, Libo Qin, Jinhao Liu, Yue Liao, Jiaqi Wang, Jinxuan Zhou, and Wanxiang Che. Rbf++: Quantifying and optimizing reasoning boundaries across measurable and unmeasurable capabilities for chain-of-thought reasoning, 2026.

[35] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models, 2026.

[36] Simin Chen, Yiming Chen, Zexin Li, Yifan Jiang, Zhongwei Wan, Yixin He, Dezhi Ran, Tianle Gu, Haizhou Li, Tao Xie, et al. Benchmarking large language models under data contamination: A survey from static to dynamic evaluation, 2025.

[37] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, et al. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026.

[38] Wanyi Chen, Xiao Yang, Xu Yang, Tianming Sha, Qizheng Li, Zhuo Wang, Bowen Xian, Fang Kong, Weiqing Liu, and Jiang Bian. Agentˆ 2 rl-bench: Can llm agents engineer agentic rl post-training?, 2026.

[39] Wenhui Chen, Jianlin Chen, Ziyao Lin, and Chi Man Vong. Auditing discovery claims: A two-sided criterion for agentic science, with the negative side decidable, 2026.

[40] Xilun Chen, Zhaleh Feizollahi, Ross Goodwin, Seungwhan Moon, Scott Yih, Pinar Donmez, Babak Damavandi, and Luna Dong. Two-level meta-rubrics for evaluating open-ended generation: Gamut, a benchmark for factual completeness, 2026.

[41] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug, 2024.

[42] Xiwei Chen. State-aware runtime for long-horizon llm agents: A conceptual framework and research agenda, 2026.

[43] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models, 2024.

[44] De Chezelles, Thibault Le Sellier, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F Xu, Siva Reddy, Quentin Cappart, et al. The browsergym ecosystem for web agent research, 2024.

[45] Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement, 2026.

[46] Gyouk Chu, Myeongho Jeon, and Eunho Yang. J-zero: Unified challenger–solver–judge co-evolution from zero data, 2026.

[47] Jeff Clune. Ai-gas: Ai-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint arXiv:1905.10985*, 2019.

[48] Benlei Cui, Ruize Wang, Junjie Li, Jinhao Chen, Longtao Huang, Yinghao Chen, Yuwen Zhai, Jingqun Tang, Ruijian Jia, Weiwei Wu, et al. Metavideoagent: Automated video-agent evolution for long-form video understanding. *arXiv preprint arXiv:2608.04587*, 2026.

[49] Tom Cunningham, Lukas Althoff, Basil Halperin, Brian Jabarian, Andrew Koh, Arjun Ramani, Phil Trammell, Parker Whitfill, and Cheryl Wu. The economics of recursive self-improvement, 2026.

[50] Zijie Dai, Siuhin He, Hui Li, Qihui Zhou, Jiajun Li, Mingcong Song, Guoping Long, Hongjie Si, Xin Yao, Lin Zhang, et al. Metis: Bridging text and code memory for self-evolving agents, 2026.

[51] Leonardo De Moura, Soonho Kong, Jeremy Avigad, Floris Van Doorn, and Jakob von Raumer. The lean theorem prover (system description). In *International conference on automated deduction*, pages 378–388. Springer, 2015.

[52] Haotian Deng, Chris Farber, Jiyoon Lee, and David Tang. Rubric-conditioned llm grading: Alignment, uncertainty, and robustness, 2025.

[53] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. volume 36, pages 28091–28114, 2023.

[54] Zihao Deng, Yining Zhu, Leiming Wang, Jingfei Lu, Junbo Wang, Chuncheng Ran, Yu Yang, Dixuan Yang, and Jikun Shen. Tree-of-experience: Hierarchical experience management for self-evolving agents, 2026.

[55] Kaustubh Dhole and Eugene Agichtein. Rubricrag: Towards interpretable and reliable llm evaluation via domain knowledge retrieval for rubric generation, 2026.

[56] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models, 2024.

[57] Ruomeng Ding, Yifei Pang, He Sun, Yizhong Wang, Zhiwei Steven Wu, and Zhun Deng. Rubrics as an attack surface: Stealthy preference drift in llm judges, 2026.

[58] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. Workarena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.

[59] Mingxuan Du, Benfeng Xu, Chiwei Zhu, Licheng Zhang, Xiaorui Wang, and Zhendong Mao. Deepresearch bench: A comprehensive benchmark for deep research agents, 2026.

[60] Yaxin Du, Xiyuan Yang, Zhifan Zhou, Wanxu Liu, Zixing Lei, Zimeng Chen, Fenyi Liu, Haotian Wu, Yuzhu Cai, Zexi Liu, et al. Datamaster: Data-centric autonomous ai research, 2026.

[61] Keyu Duan, Zichen Liu, Xin Mao, Tianyu Pang, Changyu Chen, Qiguang Chen, Michael Qizhe Shieh, and Longxu Dou. Efficient process reward model training via active learning, 2025.

[62] George B Dyson. Darwin among the machines: The evolution of global intelligence. 2012.

[63] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

[64] Damon Falck, Samer Sabri, Anja Surina, Thom Foster, Anya Sims, Sam Devlin, Dylan Rogers, Tantum Collins, Kaloyan Aleksiev, Louis Kirsch, et al. Training ai scientists to replicate research, 2026.

[65] Tianyu Fan and Chao Huang. Helix: Model-harness co-evolution for recursive self-improvement, 2026.

[66] Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, et al. Expgraph: Model-agnostic experience learning with graph-structured memory for llm agents. *arXiv preprint arXiv:2605.30712*, 2026.

[67] Tongtong Feng, Xin Wang, Zekai Zhou, Ren Wang, Yu-Wei Zhan, Guangyao Li, Qing Li, and Wenwu Zhu. Evoagent: Self-evolving agent with continual world model for long-horizon tasks, 2025.

[68] Yingchaojie Feng, Qiang Huang, Xiaoya Xie, Zhaorui Yang, Jun Yu, Wei Chen, and Anthony KH Tung. One interaction is worth a thousand guesses: Benchmarking the interactive capabilities of deep research agents, 2026.

[69] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023.

[70] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.

[71] Dazhi Fu, Jiuding Yang, Yiwen Guo, and Jicong Fan. Many voices, one reward: Multi-role rubric generation for llm judging and reward modeling, 2026.

[72] Jonathan Gabor, Jayson Lynch, and Jonathan Rosenfeld. Evilgenie: A reward hacking benchmark, 2025.

[73] Soham Gadgil, David Alexander, Sai Sunku, and Franziska Roesner. Bad memory: Evaluating prompt injection risks from memory in agentic systems, 2026.

[74] Kanishk Gandhi, Michael Y Li, Lyle Goodyear, Agam Bhatia, Louise Li, Aditi Bhaskar, Mohammed Zaman, and Noah D Goodman. Boxinggym: Benchmarking progress in automated experimental design and model discovery, 2025.

[75] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J Szostkiewicz, Dmytro Shved, Gavin J Gyimesi, Jon M Laurent, Samantha M Wright, Muhammed T Razzak, et al. A multi-agent system for automating scientific discovery. *Nature*, pages 1–3, 2026.

[76] Irving John Good. Speculations concerning the first ultraintelligent machine. In *Advances in computers*, volume 6, pages 31–88. Elsevier, 1966.

[77] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, et al. Accelerating scientific discovery with co-scientist. *Nature*, pages 1–3, 2026.

[78] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pages 57734–57811, 2024.

[79] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. Ai control: Improving safety despite intentional subversion, 2023.

[80] Xin Guan, Xiaomeng Hu, Shen Huang, Zhenyi Wang, Bo Zhang, Zijian Li, Pengjun Xie, Bo Liu, and Jiuxin Cao. Evorubric: Self-evolving rubric-driven rl for open-ended generation, 2026.

[81] Dadi Guo, Tianyi Zhou, Dongrui Liu, Chen Qian, Qihan Ren, Shuai Shao, Zhiyuan Fan, Yi R Fung, Kun Wang, Linfeng Zhang, et al. Towards self-evolving benchmarks: Synthesizing agent trajectories via test-time exploration under validate-by-reproduce paradigm, 2025.

[82] Diandian Guo, Cong Cao, Fangfang Yuan, Yingqi Wang, Yueshan Wang, and Dakui Wang. Self-authored verification is unreliable in heuristic self-improving agents, 2026.

[83] Jiacheng Guo, Suozhi Huang, Yunlong Gao, Zihao Li, Jian Ge, Xu Kuang, and Mengdi Wang. Aqua: Recursively self-improving quantitative trading research agents, 2026.

[84] Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 8154–8173, 2023.

[85] Keno Harada, Lui Yoshida, Takeshi Kojima, Yusuke Iwasawa, and Yutaka Matsuo. Automated refinement of essay scoring rubrics for language models via reflect-and-revise, 2026.

[86] Harness. Harness agenttrace. `https://www.harness.io/products/platform/agenttrace`, 2026.

[87] Helia Hashemi, Jason Eisner, Corby Rosset, Benjamin Van Durme, and Chris Kedzie. Llm-rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts, 2024.

[88] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models, 2024.

[89] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. Sia: Self improving ai with harness & weight updates, 2026.

[90] Hanhua Hong, Yizhi Li, Jiaoyan Chen, Luu Gia Huy, Sophia Ananiadou, Jung-jae Kim, and Chenghua Lin. Can llms write reliable rubrics? a meta-evaluation for experiment reproduction, 2026.

[91] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework, 2024.

[92] Yihan Hong, Huaiyuan Yao, Bolin Shen, Wanpeng Xu, Hua Wei, and Yushun Dong. From rubrics to reliable scores: Evidence-grounded text evaluation with llm judges, 2026.

[93] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(9):5149–5169, 2021.

[94] Mengkang Hu, Yao Mu, Xinmiao Yu, Mingyu Ding, Shiguang Wu, Wenqi Shao, Qiguang Chen, Bin Wang, Yu Qiao, and Ping Luo. Tree-planner: Efficient close-loop task planning with large language models, 2024.

[95] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model, 2025.

[96] Mengkang Hu, Tianxing Chen, Yude Zou, Yuheng Lei, Qiguang Chen, Ming Li, Yao Mu, Hongyuan Zhang, Wenqi Shao, and Ping Luo. Text2world: Benchmarking large language models for symbolic world model generation, 2025.

[97] Mengkang Hu, Bowei Xia, Yuran Wu, Ailing Yu, Yude Zou, Qiguang Chen, Shijian Wang, Jiarui Jin, Kexin Li, Wenxiang Jiao, et al. Agent2world: Learning to generate symbolic world models via adaptive multi-agent feedback, 2025.

[98] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025.

[99] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International conference on learning representations*, volume 2024, pages 32808–32824, 2024.

[100] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation, 2023.

[101] Siyuan Huang, Pengyu Cheng, Haotian Liu, Tao Chen, Yihao Liu, Jingwei Ni, Shijie Zhou, Ziyi Yang, Gangwei Jiang, Mengyu Zhou, et al. Skill self-play: Pushing the frontier of llm capability with co-evolving skills, 2026.

[102] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.

[103] Alex Iacob, Andrej Jovanović, William F Shen, Daniel Burkhardt, Meghdad Kurmanji, Nurbek Tastan, Lorenzo Sani, Niccolò Alberto Elia Venanzi, Ambroise Odonnat, Zeyu Cao, et al. The red queen gödel machine: Co-evolving agents and their evaluators, 2026.

[104] Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate, 2018.

[105] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, et al. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.

[106] Jiazhen Ji and Shouhong Ding. Rehearse: Stepping back from the confidence cliff in self-improving autoresearch, 2026.

[107] Hongrui Jia, Jitong Liao, Xi Zhang, Haiyang Xu, Tianbao Xie, Chaoya Jiang, Ming Yan, Si Liu, Wei Ye, and Fei Huang. Osworld-mcp: Benchmarking mcp tool invocation in computer-use agents, 2026.

[108] Wen Jiang, Mingmin Chu, Yimeng Tian, Qianxin Zhang, Haofei Yang, Rui Yang, Yang Liu, Tao Lv, and Fangming Li. Harnessevolve: Learning from reference trajectories for reliable agent self-evolution, 2026.

[109] Yixing Jiang, Kameron C Black, Gloria Geng, Danny Park, James Zou, Andrew Y Ng, and Jonathan H Chen. Medagentbench: a virtual ehr environment to benchmark medical llm agents, 2025.

[110] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.

[111] Yuxin Jin, Siyuan Zhang, Hanchen Wang, Lu Qin, Ying Zhang, and Wenjie Zhang. Exg: Self-evolving agents with experience graphs, 2026.

[112] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *Transactions of the Association for Computational Linguistics*, 12:1417–1440, 2024.

[113] Andrej Karpathy. Autoresearch. `https://github.com/karpathy/autoresearch`, 2026.

[114] Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual harness: Online adaptation for self-improving foundation agents, 2026.

[115] Jindae Kim. Reassessing one-round test-time refinement for code generation, 2026.

[116] Zae Myung Kim, Young-Jun Lee, Seungyeon Jwa, and Dongyeop Kang. Meta^n: Recursive self-improvement through emergent depth, 2026.

[117] Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. Prover-verifier games improve legibility of llm outputs, 2024.

[118] Joonho Ko, Jinheon Baek, and Sung Ju Hwang. Efficient real-time refinement of language model text generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 34548–34561, 2025.

[119] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of ai ingenuity. `https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/`, 2020.

[120] Vivek Kulkarni, Sudipta Paul, Aounon Kumar, Nicholas Tzou, and Srinivas Chappidi. Sbco: Self-supervised, verifier-grounded harness optimization for planning agents, 2026.

[121] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, et al. Rewardbench: Evaluating reward models for language modeling, 2025.

[122] Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026.

[123] Jung Hwan Lee, Kyu Ho Lee, and Gwang Hoon Yoo. Mega: Self-evolving agent optimization infrastructure via wisdom graph, 2026.

[124] Nicholas Lee, Thanakul Wattanawong, Sehoon Kim, Karttikeya Mangalam, Sheng Shen, Gopala Anumanchipalli, Michael Mahoney, Kurt Keutzer, and Amir Gholami. Llm2llm: Boosting llms with novel iterative data enhancement, 2024.

[125] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026.

[126] Chenxin Li, Zhengyang Tang, Mingxin Huang, Yunlong Lin, Shijue Huang, Shengyuan Liu, Bowen Ye, Rang Li, Lei Li, Benyou Wang, et al. Claw-eval-live: A live agent benchmark for evolving real-world workflows, 2026.

[127] Dacheng Li, Shiyi Cao, Chengkun Cao, Xiuyu Li, Shangyin Tan, Kurt Keutzer, Jiarong Xing, Joseph E Gonzalez, and Ion Stoica. S*: Test time scaling for code generation. In *EMNLP (Findings)*, pages 15964–15978, 2025.

[128] Dun Li, Jiatao Li, and Hongzhi Li. From 0-to-1 to 1-to-n: Reproducible engineering evidence for metaai recursive self-design, 2026.

[129] Fangfei Li, Chenyang Zhao, Long Wang, Feng Tian, Zhiyue Zheng, and Lv Guo. Clap: Closed-loop training, evaluation, and release control for domain agent post-training, 2026.

[130] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society, 2023.

[131] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. Llms-as-judges: a comprehensive survey on llm-based evaluation methods, 2024.

[132] Junbo Li, Boyi Liu, Canwen Xu, Yite Wang, Yuxiong He, Zhangyang Wang, Qiang Liu, and Zhewei Yao. The optimizer is the agent: Reasoning-driven search across prompts, programs, and ml workflows. *arXiv preprint arXiv:2608.06714*, 2026.

[133] Xiaoyuan Li, Keqin Bao, Moxin Li, Yubo Ma, Yichang Zhang, Wenjie Wang, Fuli Feng, and Dayiheng Liu. Ares: Automated rubric synthesis for scalable llm reinforcement learning, 2026.

[134] Gang Liao, Yujia He, Abdullah Ozturk, Zhouyang Li, Ying Wang, Zhitong Guo, Hongsen Qin, Yaobin Qin, Tao Yang, Zewei Jiang, et al. Experience graphs: The data foundation for self-improving agents, 2026.

[135] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, volume 2024, pages 39578–39601, 2024.

[136] Joy Jia Yin Lim, Xin Huang, Hao Peng, Yaxi Lu, Xin Cong, Zhong Zhang, Maosong Sun, and Yankai Lin. What is missing from ai post-training ai: An empirical analysis, 2026.

[137] Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, et al. Position: Agentic evolution is the path to evolving llms, 2026.

[138] Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents, 2026. URL `https://arxiv.org/abs/2605.30621`.

[139] Boyang Liu, Senjie Jin, Peixin Wang, Zhangyue Yin, Yibo Wang, Yuhao Zhou, Xinbing Liang, Shizheng Zhu, Yuhui Wang, Jingqi Tong, et al. Cafe: Self-improving search agents need co-evolving feedback, 2026.

[140] Changzhi Liu, Yilun Liu, Sikuan Yan, Volker Tresp, and Yunpu Ma. Mendel gödel machine: Recursive self-improving coding agents via comparative evolution, 2026.

[141] Dengcan Liu, Fengkai Yang, Xiaohan Wang, Shurui Yan, Jiajun Chai, Jiahao Li, Yikun Ban, Zhendong Mao, Wei Lin, and Guojun Yin. Cdrrm: Contrast-driven rubric generation for reliable and interpretable reward modeling, 2026.

[142] Junqi Liu, Yufan He, Yexiao He, Pengfei Guo, Dong Yang, Andriy Myronenko, Can Zhao, Hanrong Ye, Tianhao Qi, Yuyin Zhou, et al. Bat: Towards self-evolving medical research agent with stage rubrics, 2026.

[143] Tennison Liu and Mihaela van der Schaar. Truly self-improving agents require intrinsic metacognitive learning, 2025.

[144] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046, 2024.

[145] Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams, 2026.

[146] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery, 2024.

[147] Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of ai research. *Nature*, 651(8107):914–919, 2026.

[148] Shuqi Lu, Chaofan Li, Kun Luo, Zhang Zhang, Hui Wang, Hongwang Xiao, Zheng Liu, Lei Xiong, Jiahao Wang, Sen Wang, et al. Arex: Towards a recursively self-improving agent for deep research, 2026.

[149] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue, 2024.

[150] Ziming Luo, Atoosa Kasirzadeh, and Nihar B Shah. The more you automate, the less you see: Hidden pitfalls of ai scientist systems, 2025.

[151] Xiaowen Ma, Yunpu Ma, Chenyang Lin, Sikuan Yan, Jinhe Bi, Zixuan Cao, Yijun Tian, Volker Tresp, and Hinrich Schuetze. Self-evolving multi-agent systems via textual backpropagation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 9918–9951, 2026.

[152] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. volume 36, pages 46534–46594, 2023.

[153] Potsawee Manakul, Adian Liusie, and Mark Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 9004–9017, 2023.

[154] Xutao Mao, Liangjie Zhao, Xiang Zheng, and Cong Wang. Practice makes unsafe: Skill misevolution in self-improving llm agents, 2026.

[155] Nat McAleese, Rai Michael Pokorny, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. Llm critics help catch llm bugs, 2024.

[156] Fanqing Meng, Lingxiao Du, Qiguang Chen, Ziqi Zhao, Haocheng Lu, Mengkang Hu, and Michael Qizhe Shieh. Rsibench-data: Benchmarking data-centric research for recursive self-improvement, 2026.

[157] Fanqing Meng, Lingxiao Du, Zijian Wu, Guanzheng Chen, Xiangyan Liu, Jiaqi Liao, Chonghe Jiang, Zhenglin Wan, Jiawei Gu, Pengfei Zhou, et al. Clawmark: A living-world benchmark for multi-turn, multi-day, multimodal coworker agents, 2026.

[158] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants, 2024.

[159] Ning Miao, Yee Whye Teh, and Tom Rainforth. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning, 2024.

[160] Atsuyuki Miyai, Mashiro Toyooka, Takashi Otonari, Zaiying Zhao, and Kiyoharu Aizawa. Jr. ai scientist and its risk report: Autonomous scientific exploration from a baseline paper, 2025.

[161] Mohammad Mahdi Moradi, Hossam Amer, Sudhir Mudur, Weiwei Zhang, Yang Liu, and Walid Ahmed. Continuous self-improvement of large language models by test-time training with verifier-driven sample selection, 2025.

[162] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of gpt-4, 2023.

[163] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback, 2021.

[164] Siddharth Nayak, Adelmo M Orozco, Marina T Have, Vittal Thirumalai, Jackson Zhang, Darren Chen, Aditya Kapoor, Eric Robinson, Karthik Gopalakrishnan, James Harrison, et al. Long-horizon planning for multi-agent robots in partially observable environments, 2024.

[165] Michael Nguyen, Quoc Nguyen, and Paul Vuong. Recursive self-evolving agents via held-out selection, 2026.

[166] Jun Nie, Yonggang Zhang, Qianshu Cai, Yiu-ming Cheung, Xinmei Tian, and Bo Han. Evolvenet: Collaborative harness evolution for agent self-improvement, 2026.

[167] Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. Tthe: Test-time harness evolution, 2026.

[168] Tong Nie, Yuewen Mei, Yihong Tang, Junlin He, Jie Deng, Jian Sun, and Wei Ma. Evodrive: Pareto evolution for safety-critical autonomous driving via self-improving llm agents, 2026.

[169] Jingjie Ning, Xiaochuan Li, Shanshan Zhong, Ji Zeng, and Guolin Ke. Auto research for materials: Auditable ai-scientist workflows with held-out transfer, 2026.

[170] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. Code as agent harness, 2026.

[171] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[172] Stephen M Omohundro. The basic ai drives. *Artificial intelligence safety and security*, pages 47–55, 2018.

[173] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems, 2023.

[174] Tianjun Pan, Xuan Lin, Wenyan Yang, Qianyu He, Shisong Chen, Licai Qi, Wanqing Xu, Hongwei Feng, Bo Xu, and Yanghua Xiao. Rubriceval: A rubric-level meta-evaluation benchmark for llm judges in instruction following, 2026.

[175] Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. Evolving agents in the dark: Retrospective harness optimization via self-preference, 2026.

[176] Zehua Pei, Hui-Ling Zhen, Shixiong Kai, Sinno Jialin Pan, Yunhe Wang, Mingxuan Yuan, and Bei Yu. Scope: Prompt evolution for enhancing agent effectiveness, 2025.

[177] Dengyun Peng, Yuhang Zhou, Qiguang Chen, Jinhao Liu, Jingjing Chen, Libo Qin, and Wanxiang Che. Dlpo: Towards a robust, efficient, and generalizable prompt optimization framework from a deep-learning perspective., 2025.

[178] Yangda Peng, Yunjia Qi, Hao Peng, Haotian Xia, Guanzhong He, Xintong Shi, Richeng Xuan, Songyuanyi Lu, Yixian Liu, Zhichao Hu, et al. Can llm-as-a-judge reliably verify rubrics in agentic scenarios?, 2026.

[179] Pradyumna Shyama Prasad, Meiri Anto, Leon Eshuijs, Julian Moncarz, Kaustubh Kislay, and Juan J Vazquez. Baitbench: Measuring agent reward hacking with optional shortcuts planted in ml tasks, 2026.

[180] Reid Pryzant, Dan Iter, Jerry Li, Yin Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search, 2023.

[181] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 15174–15186, 2024.

[182] Libo Qin, Qiguang Chen, Xiachong Feng, Yang Wu, Yongheng Zhang, Yinghui Li, Min Li, Wanxiang Che, and Philip S Yu. Large language models meet nlp: A survey, 2026.

[183] Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, et al. Coral: Towards autonomous multi-agent evolution for open-ended discovery, 2026.

[184] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.

[185] Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. Posttrainbench: Can llm agents automate llm post-training?, 2026.

[186] Bowen Ren, Heyan Huang, Yinghao Li, and Yang Gao. Metaevo: A meta-optimization framework for experience-driven agent evolution, 2026.

[187] Shijie Ren, Xiting Wang, Meng Li, Yujie Guo, Yunhang Yao, Ziheng Peng, Xunlong Wang, Yuetan Chen, Haoyang Zhou, Yunlong Liang, et al. Self-improving large language models via progressive experience evolution. *arXiv preprint arXiv:2608.02139*, 2026.

[188] Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, RB Xiong, Qingfeng Lan, Wenyi Wang, Li Nanbo, Yibo Yang, et al. Self-improvements in modern agentic systems: A survey, 2026.

[189] Willem Röpke, Samuel Coward, Andrei Lupu, Thomas Foster, Tim Rocktäschel, and Jakob Foerster. Déjàq: Open-ended evolution of diverse, learnable and verifiable problems, 2026.

[190] Priyam Sahoo, Gaurav Mittal, Xiaomin Li, Shengjie Ma, Benjamin Steenhoek, Pingping Lin, and Yu Hu. Agentlens: Revealing the lucky pass problem in swe-agent evaluation, 2026.

[191] William Saunders, Catherine Yeh, Jeff Wu, Steven Bills, Long Ouyang, Jonathan Ward, and Jan Leike. Self-critiquing models for assisting human evaluators, 2022.

[192] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. volume 36, pages 68539–68551, 2023.

[193] Biswa Sengupta. Self-evolving agents with anytime-valid certificates, 2026.

[194] Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-r1: Learning to edit executable runtime harnesses from agent failure trajectories, 2026.

[195] Chuhan Shi, Xiaoquan Ren, Sicheng Song, Haobo Li, Rui Sheng, and Yushi Sun. Are llms ready for scientific discovery? a capability-oriented benchmark for ai scientists, 2026.

[196] Jiajun Shi, Siyuan Tao, Yuhao Wu, Zexuan Wang, Jingyuan Zhang, Jiaheng Liu, Xinping Lei, Xinrong Zhang, Siyuan Fang, Zhewen Tan, et al. S3gym: Can llms turn self-testing and self-judging into self-improvement?, 2026.

[197] Zhan Shi, Bing He, Yisi Sang, Hanqing Lu, and Benoit Dumoulin. A-evolve-training: Autonomous post-training of a 30b model, 2026.

[198] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. volume 36, pages 8634–8652, 2023.

[199] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. 2020.

[200] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

[201] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[202] Arnav Singhvi, Manish Shetty, Shangyin Tan, Christopher Potts, Koushik Sen, Matei Zaharia, and Omar Khattab. Dspy assertions: Computational constraints for self-refining language model pipelines, 2023.

[203] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. volume 35, pages 9460–9471, 2022.

[204] Jasper Snoek, Hugo Larochelle, and Ryan Adams. Practical bayesian optimization of machine learning algorithms. volume 25, 2012.

[205] Deepak Soni. Falsifiable release gates for self-improving systems: Standing invariants at scale, 2026.

[206] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: evaluating ai's ability to replicate ai research, 2025.

[207] Xiaohongshu Dots Studio and Evolvent AI. Vibelifebench: Can your life agent be proactive and persistent in a living world?, 2026. URL `https://arxiv.org/abs/2608.10875`.

[208] Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. Adaplanner: Adaptive planning from feedback with language models. volume 36, pages 58202–58245, 2023.

[209] Mengyuan Sun, Yu Li, Zhuohao Yu, Shikun Zhang, and Wei Ye. Support vector rubrics: Closing the gap between self-generated and human rubrics, 2026.

[210] Jiyuan Tan and Vasilis Syrgkanis. Causalforge: A formally grounded, self-improving agentic framework for automated research in causal inference. *arXiv preprint arXiv:2607.22511*, 2026.

[211] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, et al. Cradle: Empowering foundation agents towards general computer control, 2024.

[212] Liyan Tang, Cyrus Rashtchian, Chun-Sung Ferng, Andrew Tomkins, Da-Cheng Juan, and Tu Vu. Wikiskill: Compiling agent experience into persistent knowledge for skill evolution, 2026.

[213] Gerald Tesauro et al. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3): 58–68, 1995.

[214] Kunvar Thaman. Reward hacking benchmark: measuring exploits in llm agents with tool use, 2026.

[215] Rishabh Tiwari, Aditya Tomar, Udbhav Bamba, Monishwaran Maheswaran, Heng Yang, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. Reward under attack: Analyzing the robustness and hackability of process reward models, 2026.

[216] Jingqi Tong, Mingzhe Li, Hangcheng Li, Yongzhuo Yang, Yurong Mou, Weijie Ma, Zhiheng Xi, Hongji Chen, Xiaoran Liu, Qinyuan Cheng, et al. Ai can learn scientific taste, 2026.

[217] Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

[218] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024.

[219] Alan M Turing. Computing machinery and intelligence (1950). *Mind*, 59(236):33–60, 1987.

[220] Karthik Valmeekam, Matthew Marquez, and Subbarao Kambhampati. Can large language models really improve by self-critiquing their own plans?, 2023.

[221] Vishal Venkataramani, Haizhou Shi, Zixuan Ke, Austin Xu, Xiaoxiao He, Yingbo Zhou, Semih Yavuz, Hao Wang, and Shafiq Joty. Mas-prove: Understanding the process verification of multi-agent systems, 2026.

[222] Vansh Wahi. Llm-as-a-judge is not an oracle: Why self-improving agents need deterministic guardrails, 2026.

[223] Congchao Wang, Diwakar Singh, Qiaozi Gao, Spyros Matsoukas, Yang Liu, and Mahdi Namazifar. Reasoning jury: Multi-model consensus for evaluating reasoning traces, 2026.

[224] Danqing Wang and Lei Li. Learning from mistakes via cooperative study assistant for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10667–10685, 2023.

[225] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[226] Qinsi Wang, Jing Shi, Huazheng Wang, Kun Wan, Yiran Wu, Bo Liu, Qingyun Wu, Hai Helen Li, Yiran Chen, Handong Zhao, et al. From rlvr to rlsvr: Task transformation induces self-verifiable rewards for open-ended llm self-improvement. *arXiv preprint arXiv:2607.23802*, 2026.

[227] Rui Wang, Joel Lehman, Aditya Rawal, Jiale Zhi, Yulun Li, Jeffrey Clune, and Kenneth Stanley. Enhanced poet: Open-ended reinforcement learning through unbounded invention of learning challenges and their solutions. In *International conference on machine learning*, pages 9940–9951. PMLR, 2020.

[228] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. Scienceworld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11279–11298, 2022.

[229] Xing Wang, Xiaoyi Zhang, and Jie Shao. Auditing harness tampering in self-improving agents, 2026.

[230] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents, 2025.

[231] Xuefei Julie Wang, Lauren Hyoseo Yoon, Chengrui Qu, Amanda Zichang Wang, Atharva Sehgal, Eric Mazumdar, and Yisong Yue. Knowledge-centric self-improvement, 2026.

[232] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. 2022.

[233] Yanbo Wang, Jinhua Hao, Yuze Shi, Kun Yuan, and Ming Sun. No time like the present: Agentic test-time training for llm agents, 2026.

[234] Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, 2026.

[235] Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. Evoagentx: An automated framework for evolving agentic workflows. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 643–655, 2025.

[236] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions, 2023.

[237] Yuru Wang, Lejun Cheng, Yuxin Zuo, Sihang Zeng, Bingxiang He, Che Jiang, Junlin Yang, Yuchong Wang, Kaikai Zhao, Weifeng Huang, et al. Naturebench: Can coding agents match the published sota of nature-family papers?, 2026.

[238] Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. Metaskill-evolve: Recursive self-improvement of llm agents via two-timescale meta-skill evolution, 2026.

[239] Zhiyuan Wang, Shengcai Liu, Jiahao Wu, Ning Lu, Hui Ouyang, Shaofeng Zhang, Haoze Lv, and Ke Tang. Cooperative coevolution for resource-constrained agentic llm post-training, 2026.

[240] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Shawn Ma, and Yitao Liang. Describe, explain, plan and select: interactive planning with llms enables open-world multi-task agents. volume 36, pages 34153–34189, 2023.

[241] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. volume 35, pages 24824–24837, 2022.

[242] Tianxin Wei, Zhan Shi, Minhua Lin, Bing He, Zewen Liu, Yisi Sang, Yuanchen Bei, Xuying Ning, Jiaru Zou, Ting-Wei Li, et al. Evo-harness: Context-to-harness skill compilation for self-evolving agents. *arXiv preprint arXiv:2608.15071*, 2026.

[243] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, et al. Livebench: A challenging, contamination-limited llm benchmark, 2025.

[244] Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts, 2024.

[245] Chen Henry Wu and Aditi Raghunathan. Self-trained verification for training-and test-time self-improvement, 2026.

[246] Juntao Wu, Wei Wen, Xianting Huang, Shuai Pang, Ruizhi Qiao, Xing Sun, and Ke Wang. Breaking the evaluation paradox: Evaluating high-entropy search with computationally irreducible constraints, 2026.

[247] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.

[248] Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason E Weston, and Sainbayar Sukhbaatar. Meta-rewarding language models: Self-improving alignment with llm-as-a-meta-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11548–11565, 2025.

[249] Xiaodong Wu, Yu Shi, Qi Li, Zhimin Zhao, Xiangman Li, Bram Adams, Ahmed E Hassan, and Jianbing Ni. Evomal: Self-poisoning in self-evolving coding agents, 2026.

[250] Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin, Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin Xu, Jia Li, and Jian Guo. Bayesian-agent: Posterior-guided skill evolution for llm agent harnesses. *arXiv preprint arXiv:2606.08348*, 2026.

[251] Yangzhen Wu, Aaron J Li, Wenjie Ma, Li Cao, Ziheng Zhou, Mert Cemri, Shu Liu, Yuran Xiu, Chenxiao Yan, Haikun Zhao, et al. Benchevolver: Frontier task synthesis via solution-centric evolution, 2026.

[252] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026.

[253] Yang Xiao, Yusong Sun, Haoyi Wu, Wenyang Hui, Wen Da, Zhaokai Luo, Mu Chuan, Yao Hu, Wenjie Li, and Chengyue Jiang. Pilot in the loop: Live self-improvement for long-horizon agents, 2026.

[254] Qiujie Xie, Yixuan Weng, Minjun Zhu, Fuchen Shen, Shulin Huang, Zhen Lin, Jiahui Zhou, Zilan Mao, Zijie Yang, Linyi Yang, et al. How far are ai scientists from changing the world?, 2025.

[255] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.

[256] Weichu Xie, Haozhe Zhao, Wenpu Liu, Yongfu Zhu, Liang Chen, Minghao Ye, Zirong Chen, Yuqi Xu, Shuai Dong, Ziyue Wang, et al. Step-wise rubric rewards for llm reasoning, 2026.

[257] Lei Xiong, Kun Luo, Ziyi Xia, Wenbo Zhang, Jin-Ge Yao, Zheng Liu, Jingying Shao, Jianlyu Chen, Hongjin Qian, Xi Yang, et al. Autoresearchbench: Benchmarking ai agents on complex scientific literature discovery. *arXiv preprint arXiv:2604.25256*, 2026.

[258] Siheng Xiong, Ali Payani, Oguzhan Gungordu, and Faramarz Fekri. Dive: Unlocking self-improvement in frozen language models through diversity-driven skill evolution, 2026.

[259] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts llm agents: An empirical study of experience-following behavior, 2026.

[260] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions, 2023.

[261] Cheng Xu, Nan Yan, Liming Chen, M Kechadi, et al. Phantom gains: Auditing self-improvement against a measured null, 2026.

[262] Wanghan Xu, Shuo Li, Tianlin Ye, Qinglong Cao, Yixin Chen, Hengjian Gao, Yiheng Wang, Qi Li, Kun Li, Sheng Xu, et al. Researchclawbench: A benchmark for end-to-end autonomous scientific research, 2026.

[263] Weilu Xu, Yunzhi Shen, Xinye Wang, Ranfei Dang, and Shujian Huang. Rubric-as-experts: Case-specific mqm rubrics for translation quality evaluation, 2026.

[264] Yuanyuan Xu, Wenjie Zhang, Yin Chen, Xuemin Lin, and Ying Zhang. Self-evolving agents as dynamic graph transformation: A survey and new perspective, 2026.

[265] Hui Xue and Fan Yang. Rethinking self-evolving agents: Do we still need prescribed optimization pipelines?, 2026.

[266] Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, Yuxin Chen, Mengdi Wang, and Ling Yang. Past-bench: Benchmarking the foundations of recursive self-improvement in personal agents. *arXiv preprint arXiv:2608.04003*, 2026.

[267] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025.

[268] Dong Yan, Jian Liang, Dapeng Hu, Ran He, Nicholas Jing Yuan, Qi Zhang, and Tieniu Tan. Agentstream: How well do self-evolving llm agents perform under streaming tasks?, 2026.

[269] Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Shuo Chen, Zhankui He, Noveen Sachdeva, Weili Wang, Ed H Chi, et al. Pacevolve++: Improving test-time learning for evolutionary search agents, 2026.

[270] Zheng Yan, Jingxiang Weng, Charles Chen, Dengyun Peng, Ethan Qin, Jiannan Guan, Jinhao Liu, Qiming Yu, Yixin Yuan, Fanqing Meng, et al. Do coding agents understand least-privilege authorization?, 2026.

[271] Cehao Yang, Xiaojun Wu, Xueyuan Lin, Chengjin Xu, Xuhui Jiang, Hui Xiong, and Jian Guo. Datafoundry: Evolving data preparators via recursive self-improvement, 2026.

[272] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068, 2024.

[273] Haocheng Yang, Licheng Pan, Xiaoxi Li, Zhichao Chen, Zhiheng Zhang, Yuan Lu, Haoxuan Li, and Hao Wang. Rubrics on trial: Evolving rubrics from a single query via synthetic pairwise evidence, 2026.

[274] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. Intercode: Standardizing and benchmarking interactive coding with execution feedback, 2023.

[275] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024.

[276] Junlin Yang, Che Jiang, Yu Fu, Tianwei Luo, Can Ren, Weizhi Wang, Kaikai Zhao, Hongyi Liu, Yuxin Zuo, Yuru Wang, et al. Frontis-ma1: Training an ai4ai model towards recursive self-improvement in machine learning engineering. *arXiv preprint arXiv:2607.28568*, 2026.

[277] Min Yang, Jinghua Piao, Xu Xia, Xiaochong Lan, Jiaju Chen, Yongshun Gong, and Yong Li. Skillmaster: Toward autonomous skill mastery in llm agents, 2026.

[278] Minglai Yang, Xinyu Guo, Utkarsh Tyagi, Mian Zhang, Razvan Dumitru, Sunjie Hou, Yunzhong He, Daniel Yue Zhang, and Ying Liu. Rubric dropout: A simple way to mitigate reward hacking in rubric-as-reward rl, 2026.

[279] Ruihan Yang, Fanghua Ye, Jian Li, Siyu Yuan, Zhaopeng Tu, Xiaolong Li, Deqing Yang, et al. The lighthouse of language: Enhancing llm agents via critique-guided improvement. volume 38, pages 164647–164678, 2026.

[280] Xiyuan Yang, Sheikh Sarwar, Jingru Cheng, Zhan Shi, Duanshun Li, Huiyuan Chen, Haiyang Zhang, Chenlei Guo, Jingrui He, and Zhenyu Liao. Scaling automatic research agents via world models. *arXiv preprint arXiv:2608.12564*, 2026.

[281] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, et al. Skillopt: Executive strategy for self-evolving agent skills, 2026.

[282] Zonglin Yang, Xingtong Liu, and Xinyan Xu. Sciintegrity-bench: A benchmark for evaluating academic integrity in ai scientist systems. *arXiv preprint arXiv:2605.10246*, 2026.

[283] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. 2022.

[284] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. volume 36, pages 11809–11822, 2023.

[285] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL `https://arxiv.org/abs/2406.12045`.

[286] Qinyuan Ye, Yu Li, Yada Pruksachatkun, Jiaxin Zhang, and Chien-Sheng Wu. On the fragility of self-improving agents: Variance, task order, and underspecification, 2026.

[287] Wenqian Ye, Ziwei Guan, Eric Xie, Bohan Liu, Shivani Modi, Buyun Zhang, Ellie Dingqiao Wen, Henry Kautz, and Aidong Zhang. Proofevolve: Neuro-symbolic evolution for formal automated theorem proving, 2026.

[288] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, 2025.

[289] Zonghao Ying, Xiangfan Wu, Huiyu Wu, Xing Zheng, Huangsheng Cheng, Xiaorong Shi, and Jing Guo. Skilljack: Persistent skill backdoors in self-evolving agents, 2026.

[290] Takuma Yoneda, Jiading Fang, Peng Li, Huanyu Zhang, Tianchong Jiang, Shengjie Lin, Ben Picker, David Yunis, Hongyuan Mei, and Matthew R Walter. Statler: State-maintaining language models for embodied reasoning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15083–15091. IEEE, 2024.

[291] Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. Assistantbench: Can web agents solve realistic and time-consuming tasks?, 2024.

[292] Kotaro Yoshida, So Kuroki, Yuki Imajuku, Taishi Nakamura, Ryunosuke Iwai, Haruki Goda, and Takuya Akiba. Feedback-to-rubrics: Can we learn expert criteria from inline comments?, 2026.

[293] Lui Yoshida. Do we need a detailed rubric for automated essay scoring using large language models?, 2025.

[294] Zhaochen Yu, Yingcheng Wu, Zhenfei Yin, Kaiyuan Chen, Zhe Zhao, Mengdi Wang, Shuicheng Yan, and Ling Yang. Recursive experiential-working memory evolution for long-horizon agent harnesses, 2026.

[295] Zhaojian Yu, Penghao Yin, Shuzheng Gao, Shilin He, Kai Cai, and Xiao-Ping Zhang. Autotrainess: Teaching language models to improve language models autonomously, 2026.

[296] Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024.

[297] Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. Agent-r: Training language model agents to reflect via iterative self-training, 2025.

[298] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2024.

[299] Eliezer Yudkowsky et al. Artificial intelligence as a positive and negative factor in global risk. volume 1, page 184. New York, 2008.

[300] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic" differentiation" via text, 2024.

[301] Kamer Ali Yuksel, Thiago Castro Ferreira, Mohamed Al-Badrashiny, and Hassan Sawaf. A multi-ai agent system for autonomous optimization of agentic ai solutions via iterative refinement and llm-driven feedback loops. In *Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)*, pages 52–62, 2025.

[302] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning, 2022.

[303] Yuhao Zhan, Bingxiang He, Zecong Tang, and Chaojun Xiao. Pace-bench: Benchmarking physics adaptation via code evolution in dynamic environments, 2026.

[304] Di Zhang. Agentdevel: Reframing self-evolving llm agents as release engineering, 2026.

[305] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, volume 2026, pages 104223–104294, 2026.

[306] Jiang Zhang, Bing Yuan, and Qian Zhang. Self-reference in large language models: The introspection threshold for recursive self-improvement, 2026.

[307] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation, 2025.

[308] Jiayi Zhang, Yongfeng Gu, Jianhao Ruan, Maojia Song, Yiran Peng, Zhiguang Han, Jinyu Xiang, Zhitao Wang, Caiyin Yang, Yixi Ouyang, et al. Harnessing agentic evolution, 2026.

[309] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, et al. Swe-bench goes live!, 2026.

[310] Lunjun Zhang, Ryan Chen, and Bradly C Stadie. Evolutionary system prompt learning for reinforcement learning in llms, 2026.

[311] Qiyuan Zhang, Junyi Zhou, Yufei Wang, Fuyuan Lyu, Yidong Ming, Can Xu, Qingfeng Sun, Kai Zheng, Peng Kang, Xue Liu, et al. Rubricbench: Aligning model-generated rubrics with human standards, 2026.

[312] Tianjun Zhang, Aman Madaan, Luyu Gao, Steven Zheng, Swaroop Mishra, Yiming Yang, Niket Tandon, and Uri Alon. In-context principle learning from mistakes, 2024.

[313] Wanpeng Zhang and Zongqing Lu. Adarefiner: Refining decisions of language models with adaptive feedback. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 782–799, 2024.

[314] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. The blind curator: How a biased judge silently disables skill retirement in self-evolving agents, 2026.

[315] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries, 2026.

[316] Xing Zhang, Guanghui Wang, Yanwei Cui, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Who grades the grader? co-evolving evaluation metrics and skills for self-improving llm agents, 2026.

[317] Yifan Zhang, Yutong Dai, Juntao Tan, Luyu Yang, Rishi Mullur, Thai Hoang, Zhiyuan Hu, James Zhu, Phil Mui, Silvio Savarese, et al. Darwinx: Evolving agent harnesses through natural selection. *arXiv preprint arXiv:2608.07545*, 2026.

[318] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. 2022.

[319] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024.

[320] Andrew Zhao, Yiran Wu, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. volume 38, pages 105816–105879, 2026.

[321] Zhengyang Zhao, Shengjie Ye, Lu Ma, Hao Liang, Hengyi Feng, and Wentao Zhang. Andes: Agent native data evolving synthesis tool for autonomous instruction alignment, 2026.

[322] Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. Seagym: An evaluation environment for self-evolving llm agents, 2026.

[323] Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. Lifelongagentbench: Evaluating llm agents as lifelong learners, 2025.

[324] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. volume 36, pages 46595–46623, 2023.

[325] Hailin Zhong and Shengxin Zhu. Ai harness engineering: A runtime substrate for foundation-model software agents, 2026.

[326] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models, 2023.

[327] Junting Zhou, Jin Chen, Linfeng Hao, Denghui Cao, Zheyu Wang, Qiguang Chen, Chaoyou Fu, Jiaze Chen, Yuchen Wu, Ge Zhang, et al. Babe: Biology arena benchmark, 2026.

[328] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents, 2024.

[329] Tailin Zhou. Hierarchical self-improvement: A framework for task-specific evolvable agent harnesses, 2026.

[330] Yuhang Zhou, Lizhu Zhang, Yifan Wu, Jiayi Liu, Xiangjun Fan, Zhuokai Zhao, and Hong Yan. Synthetic sandbox for training machine learning engineering agents, 2026.

[331] Yuhang Zhou, Kai Zheng, Qiguang Chen, Mengkang Hu, Qingfeng Sun, Can Xu, and Jingjing Chen. Offseeker: Online reinforcement learning is not all you need for deep research agents, 2026.

[332] Minjun Zhu, Qiujie Xie, Yixuan Weng, Jian Wu, Zhen Lin, Linyi Yang, and Yue Zhang. Ai scientists fail without strong implementation capability, 2025.

[333] Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. 2016.

[334] Adam Zweiger, Jyo Pari, Han Guo, Yoon Kim, and Pulkit Agrawal. Self-adapting language models, 2026.